



Data Science - Project Course

Lecture 5: Nonlinear Regression Analysis

Tanujit Chakraborty

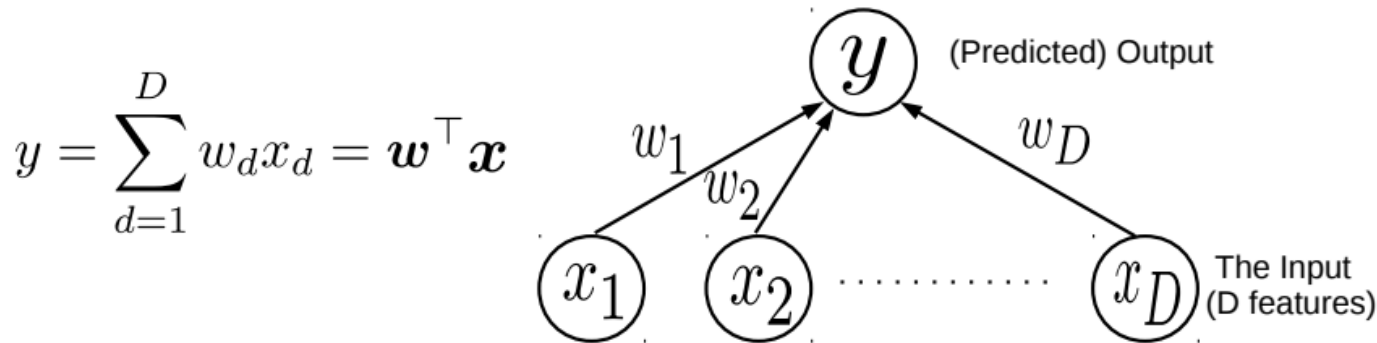
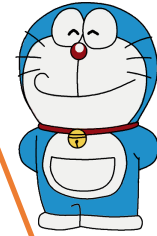
@ Sorbonne

tanujitisi@gmail.com

Linear Models



- Consider learning to map an input $\mathbf{x} \in \mathbb{R}^D$ to the corresponding (say real-valued) output y
- Assume the output to be a linear weighted combination of the D input features



This defines a linear model with D parameters given by a “**weight vector**” $\mathbf{w} = [w_1, w_2, \dots, w_D]$

Each of these weights have a simple interpretation: w_d is the “**weight**” or importance of the d^{th} feature in making this prediction

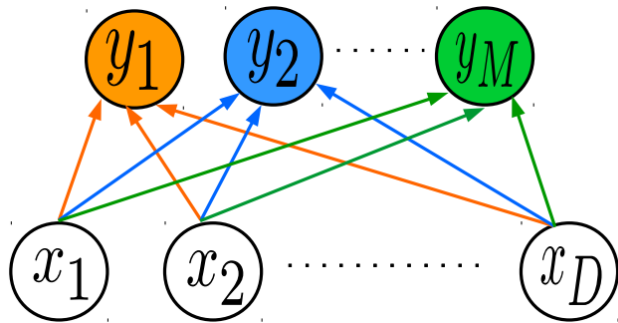
- This simple model can be used for **Linear Regression**

The “optimal” weights are unknown and have to be learned by solving an **optimization problem**, using some **training data**

- This simple model can also be used as a “**building block**” for more complex models
 - Even classification (binary/multi-class/multi-output/multi-label) and various other ML/DL models.
 - Even unsupervised learning problems (e.g., dimensionality reduction models)

Simple Linear Models as Building Blocks

- In some regression problems, each output itself is a real-valued vector $\mathbf{y} \in \mathbb{R}^M$
 - Example: Given a full body image of a person, predict height, weight, hand size, and leg size ($M = 4$)
- Such problems are commonly known as **multi-output regression**
- We can assume a separate linear model for each of the M outputs y_m ($m = 1, 2, \dots, M$)



$$y_m = \mathbf{w}_m^T \mathbf{x}$$

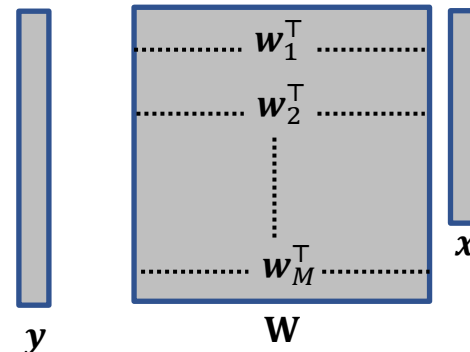
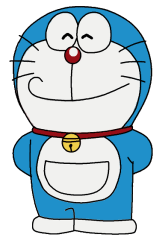
Now each $\mathbf{w}_m \in \mathbb{R}^D$ is a D -dim weight vector for predicting the m^{th} output

$$\mathbf{y} = \mathbf{W} \mathbf{x}$$

Here $\mathbf{W}$ is an $M \times D$ **weight matrix** with its m^{th} row containing $\mathbf{w}_m$

Note: Learning M separate models may not be ideal these multiple outputs are somewhat correlated with each other.

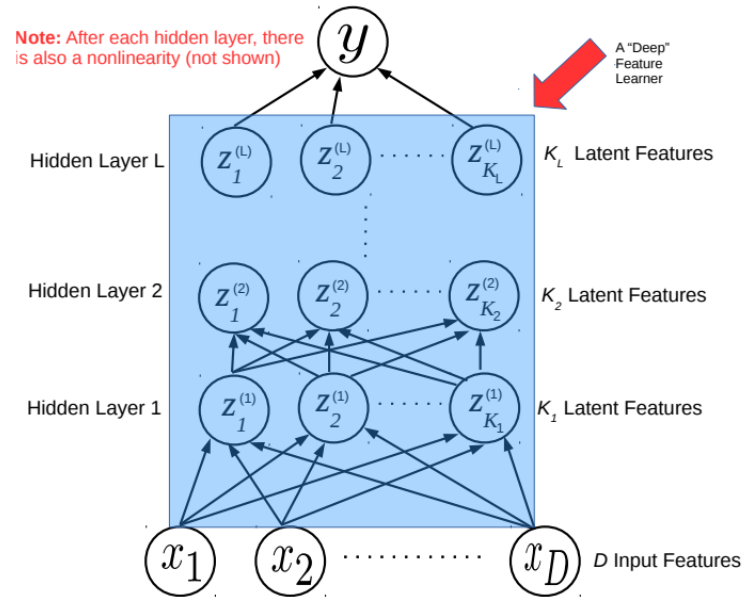
But this model can be extended to handle such situation (techniques are a bit advanced to be discussed right now – but if curious, you may look up more about **multitask learning** techniques. [Link](#))



Learning this model will require us to learn this weight matrix (or equivalently, the M weight vectors)

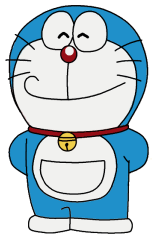
Simple Linear Models as Building Blocks

- Linear models are building blocks for even **deep learning** model (each layer is like a multi-output linear model, followed by a nonlinearity)

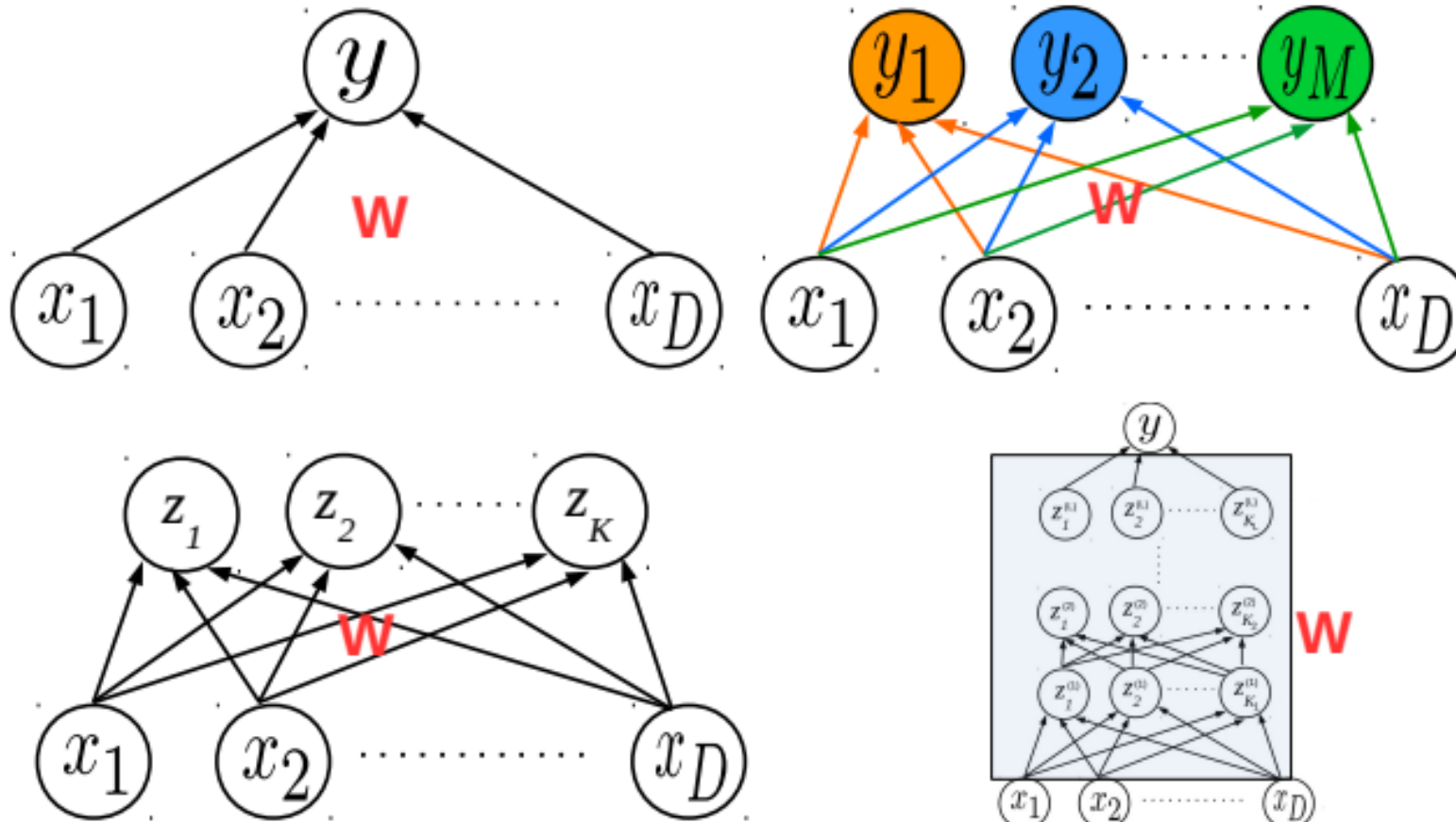


In a deep learning model, each layer learns a latent feature representation of the inputs using a model like a multi-output linear model, followed by a nonlinearity

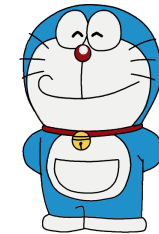
The last (output) layer can have one or more outputs
More on this when we discuss deep learning later.



Learning Linear Models



Linear Models are ubiquitous!
How do we learn them from data?



For linear models, **learning = Learning the model parameters (the weights)**

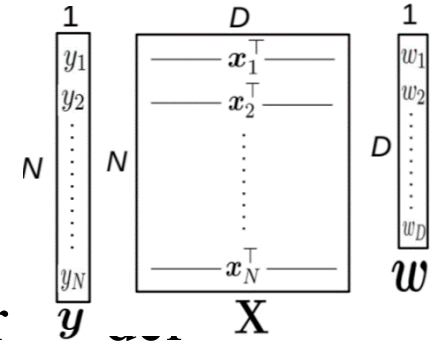
We will formulate learning as an optimization problem w.r.t. these parameters.

Linear Regression

- Given: Training data with N input-output pairs $\{(\mathbf{x}_n, y_n)\}_{n=1}^N$, $\mathbf{x}_n \in \mathbb{R}^D$, $y_n \in \mathbb{R}$

- Goal: Learn a model to predict the output for new test inputs

- Assume the function that approximates the I/O relationship to be a linear



$$y_n \approx f(\mathbf{x}_n) = \mathbf{w}^\top \mathbf{x}_n \quad (n = 1, 2, \dots, N)$$

Can also write all of them compactly using matrix-vector notation as $\mathbf{y} \approx \mathbf{X}\mathbf{w}$

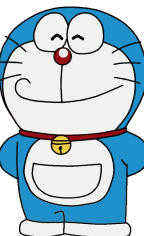
- Let's write the total error or "loss" of this model over the training data as

So, goal of learning is to find the $\mathbf{w}$ that **minimizes this loss** + **does well on test data right?**



$$L(\mathbf{w}) = \sum_{n=1}^N \ell(y_n, \mathbf{w}^\top \mathbf{x}_n)$$

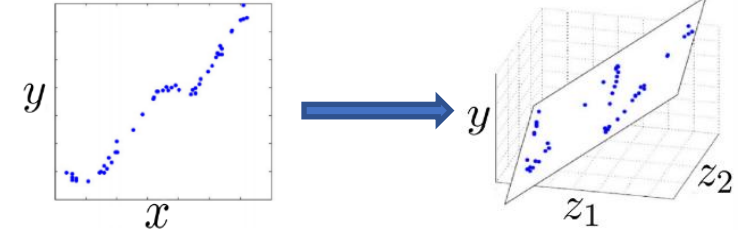
$\ell(y_n, \mathbf{w}^\top \mathbf{x}_n)$ measures the prediction error or "loss" or "deviation" of the model on a single training input $(\mathbf{x}_n, y_n)$



Linear Regression: Pictorially

- Linear regression is like fitting a line or (hyper)plane to a set of points

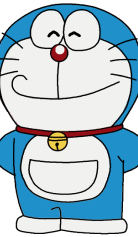
$$\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \phi(x)$$



Original (single) feature
Nonlinear curve needed

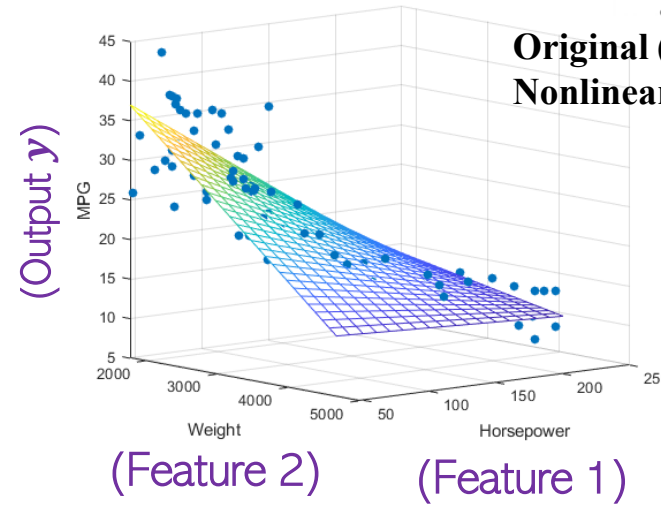
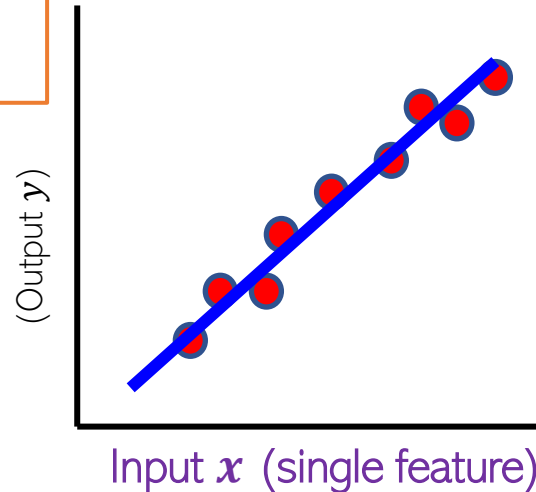
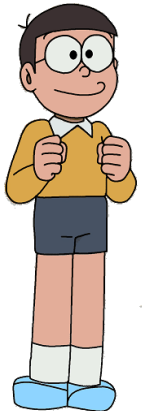
Two features
Can fit a plane (linear)

No. We can even fit a curve using a linear model after suitably transforming the inputs $y \approx \mathbf{w}^T \phi(x)$



What if a line/plane doesn't model the input-output relationship very well, e.g., if their relationship is better modeled by a nonlinear curve or curved surface?

Do linear models become useless in such cases?



- The line/plane must also predict outputs the unseen (test) inputs well !

Linear Regression with Squared Loss

- In this case, the loss function will be

In matrix-vector notation, can write it compactly as $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 = (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w})$

$$L(\mathbf{w}) = \sum_{n=1}^N (y_n - \mathbf{w}^\top \mathbf{x}_n)^2$$

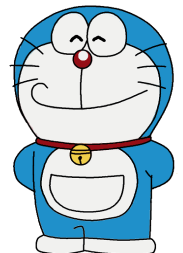
- Let us find the $\mathbf{w}$ that optimizes (minimizes) the above squared loss
- We need calculus and optimization to do this!
- The LS problem can be solved easily and has a **closed form** solution

The “least squares” (LS) problem Gauss-Legendre (1805-1809)

$$\mathbf{w}_{LS} = \arg \min_{\mathbf{w}} L(\mathbf{w}) = \arg \min_{\mathbf{w}} \sum_{n=1}^N (y_n - \mathbf{w}^\top \mathbf{x}_n)^2$$

$$\mathbf{w}_{LS} = \left(\sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^\top \right)^{-1} \left(\sum_{n=1}^N y_n \mathbf{x}_n \right) = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Closed form solutions to ML problems are rare.



Proof: (more on this later)

- We wanted to find the minima of $L(\mathbf{w}) = \sum_{n=1}^N (y_n - \mathbf{w}^\top \mathbf{x}_n)^2$
- Let us apply basic rule of calculus: Take first derivative of $L(\mathbf{w})$ and set to zero

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \frac{\partial}{\partial \mathbf{w}} \sum_{n=1}^N (y_n - \mathbf{w}^\top \mathbf{x}_n)^2 = \sum_{n=1}^N 2(y_n - \mathbf{w}^\top \mathbf{x}_n) \frac{\partial}{\partial \mathbf{w}} (y_n - \mathbf{w}^\top \mathbf{x}_n) = 0$$

Chain rule of calculus

Partial derivative of dot product w.r.t each element of $\mathbf{w}$

Result of this derivative is $\mathbf{x}_n$ - same size as $\mathbf{w}$

- Using the fact $\frac{\partial}{\partial \mathbf{w}} \mathbf{w}^\top \mathbf{x}_n = \mathbf{x}_n$, we get $\sum_{n=1}^N 2(y_n - \mathbf{w}^\top \mathbf{x}_n) \mathbf{x}_n = 0$
- To separate $\mathbf{w}$ to get a solution, we write the above as

$$\sum_{n=1}^N 2\mathbf{x}_n (y_n - \mathbf{x}_n^\top \mathbf{w}) = 0 \quad \longrightarrow \quad \sum_{n=1}^N y_n \mathbf{x}_n - \mathbf{x}_n \mathbf{x}_n^\top \mathbf{w} = 0$$

$$\mathbf{w}_{LS} = (\sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^\top)^{-1} (\sum_{n=1}^N y_n \mathbf{x}_n) = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Problem(s) with the Solution!

- We minimized the objective $L(\mathbf{w}) = \sum_{n=1}^N (y_n - \mathbf{w}^\top \mathbf{x}_n)^2$ w.r.t. $\mathbf{w}$ and got

$$\mathbf{w}_{LS} = (\sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^\top)^{-1} (\sum_{n=1}^N y_n \mathbf{x}_n) = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

- Problem: **The matrix $\mathbf{X}^\top \mathbf{X}$ may not be invertible?**

- This may lead to non-unique solutions for $\mathbf{w}_{opt}$

- Problem: **Overfit since we only minimized loss defined on training data?**

- Weights $\mathbf{w} = [w_1, w_2, \dots, w_D]$ may become arbitrarily large to fit training data perfectly
- Such weights may perform poorly on the test data however

$R(\mathbf{w})$ (called the **Regularizer**) measures the “magnitude” of $\mathbf{w}$

- One Solution: Minimize a **regularized objective** $L(\mathbf{w}) + \lambda R(\mathbf{w})$

- The reg. will prevent the elements of $\mathbf{w}$ from becoming too large
- Reason: Now we are minimizing **training error** + **magnitude of vector $\mathbf{w}$**

$\lambda \geq 0$ is the **reg. hyperparam.**
Controls how much we wish to regularize (needs to be tuned via cross-validation)

Regularized Least Squares (a.k.a. Ridge Regression)



- Recall that the regularized objective is of the form $L_{reg}(\mathbf{w}) = L(\mathbf{w}) + \lambda R(\mathbf{w})$
- One possible/popular regularizer: the squared Euclidean (ℓ_2 squared) norm of $\mathbf{w}$
- With this regularizer, we have the regularized least squares problem as

$$R(\mathbf{w}) = \|\mathbf{w}\|_2^2 = \mathbf{w}^\top \mathbf{w}$$

- Proceeding just like the LS case, we can find the optimal $\mathbf{w}$ which is given by

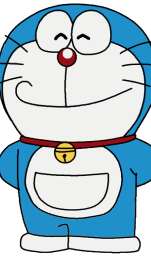
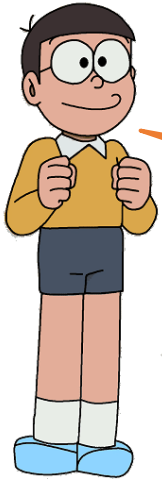
$$\mathbf{w}_{ridge} = \arg \min_{\mathbf{w}} L(\mathbf{w}) + \lambda R(\mathbf{w}) = \arg \min_{\mathbf{w}} \sum_{n=1}^N (y_n - \mathbf{w}^\top \mathbf{x}_n)^2 + \lambda \mathbf{w}^\top \mathbf{w}$$

Why is the method called “ridge” regression?

Look at the form of the solution. We are adding a small value λ to the diagonals of the $D \times D$ matrix $\mathbf{X}^\top \mathbf{X}$ (like adding a ridge/mountain to some land)

$$\mathbf{w}_{ridge} = (\sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^\top + \lambda \mathbf{I}_D)^{-1} (\sum_{n=1}^N y_n \mathbf{x}_n) = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_D)^{-1} \mathbf{X}^\top \mathbf{y}$$

How to prove it?



Regularized Least Squares (a.k.a. Ridge Regression)



Thus, minimize $e'e + \lambda \beta' \beta$ (in matrix form).

$$\begin{aligned} R(\beta) &= \text{RSS}(\beta) = (Y - X\beta)'(Y - X\beta) + \lambda \beta' \beta \\ &= (Y' - \beta' X')(Y - X\beta) + \lambda \beta' \beta \\ &= Y'Y - \beta' X'Y - Y'X\beta + \beta' X'X\beta + \lambda \beta' \beta \end{aligned}$$

$$\frac{\partial R(\beta)}{\partial \beta} = -2X'Y + 2\beta X'X + 2\lambda \beta$$

$$\text{Now, } \frac{\partial R(\beta)}{\partial \beta} = 0$$

$$\Rightarrow (X'X + \lambda I)\beta = X'Y$$

$$\Rightarrow \hat{\beta}_{\text{Ridge}} = (X'X + \lambda I)^{-1} X'Y$$

A closer look at ℓ_2 regularization



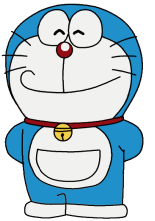
- The regularized objective we minimized is

$$L_{reg}(\mathbf{w}) = \sum_{n=1}^N (y_n - \mathbf{w}^\top \mathbf{x}_n)^2 + \lambda \mathbf{w}^\top \mathbf{w}$$

- Minimizing $L_{reg}(\mathbf{w})$ w.r.t. $\mathbf{w}$ gives a solution for $\mathbf{w}$ that

- Keeps the training error small
- Has a small ℓ_2 squared norm $\mathbf{w}^\top \mathbf{w} = \sum_{d=1}^D w_d^2$

Remember – in general, weights with large magnitude are bad since they can cause overfitting on training data and may not work well on test data



Good because, consequently, the individual entries of the weight vector $\mathbf{w}$ are also prevented from becoming too large

Not a “smooth” model since its test data predictions may change drastically even with small changes in some feature’s value

- Small entries in $\mathbf{w}$ are good since they lead to “smooth” models

$\mathbf{x}_n =$	1.2	0.5	2.4	0.3	0.8	0.1	0.9	2.1
$\mathbf{x}_m =$	1.2	0.5	2.4	0.3	$0.8 + \epsilon$	0.1	0.9	2.1

Exact same feature vectors only differing in just one feature by a small amount

$$y_n = 0.8$$

$$y_m = 100$$

Very different outputs though (maybe one of these two training ex. is an outlier)

A typical $\mathbf{w}$ learned without ℓ_2 reg.

3.2	1.8	1.3	2.1	10000	2.5	3.1	0.1
-----	-----	-----	-----	-------	-----	-----	-----

Just to fit the training data where one of the inputs was possibly an outlier, this weight became too big. Such a weight vector will possibly do poorly on normal test inputs

Other Ways to Control Overfitting



- Use a regularizer $R(\mathbf{w})$ defined by other norms, e.g.,

ℓ_1 norm regularizer

$$\|\mathbf{w}\|_1 = \sum_{d=1}^D |w_d|$$

When should I use these regularizers instead of the ℓ_2 regularizer?

$$\|\mathbf{w}\|_0 = \#\text{nnz}(\mathbf{w})$$

ℓ_0 norm regularizer (counts number of non-zeros in $\mathbf{w}$)

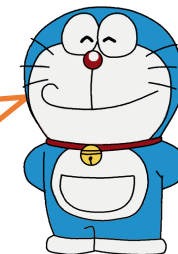
Automatic feature selection? Wow, cool!!! But how exactly?

Note that optimizing loss functions with such regularizers is usually harder than ridge reg. but several advanced techniques exist (we will see some of those later)

Use them if you have a very large number of features but many irrelevant features. These regularizers can help in **automatic feature selection** (e.g., **LASSO regression**)

Using such regularizers gives a **sparse** weight vector $\mathbf{w}$ as solution

sparse means many entries in $\mathbf{w}$ will be zero or near zero. Thus those features will be considered irrelevant by the model and will not influence prediction



Linear Regression as Solving System of Linear Equations



- The form of the lin. reg. model $\mathbf{y} \approx \mathbf{X}\mathbf{w}$ is akin to a system of linear equation
- Assuming N training examples with D features each, we have

First training example: $y_1 = x_{11}w_1 + x_{12}w_2 + \dots + x_{1D}w_D$

Second training example: $y_2 = x_{21}w_1 + x_{22}w_2 + \dots + x_{2D}w_D$

⋮

N-th training example: $y_N = x_{N1}w_1 + x_{N2}w_2 + \dots + x_{ND}w_D$

Note: Here x_{nd} denotes the d^{th} feature of the n^{th} training example

N equations and D unknowns here ($w_1, w_2, \dots, w_D$)

- However, in regression, we rarely have $N = D$ but rather $N > D$ or $N < D$
 - Thus, we have an **underdetermined** ($N < D$) or **overdetermined** ($N > D$) system
 - Methods to solve over/underdetermined systems can be used for lin-reg as well
 - Many of these methods don't require expensive matrix inversion

Now solve this!

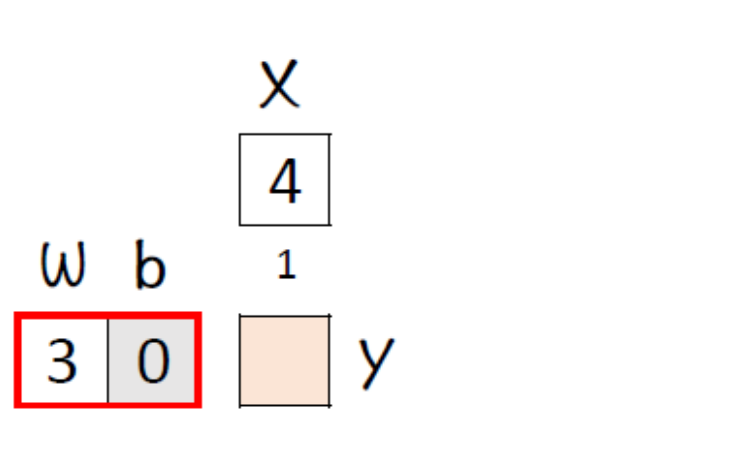
Solving linear reg.
as system of linear eq.

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \longrightarrow \boxed{\mathbf{A}\mathbf{w} = \mathbf{b}} \text{ where } \mathbf{A} = \mathbf{X}^\top \mathbf{X}, \text{ and } \mathbf{b} = \mathbf{X}^\top \mathbf{y}$$

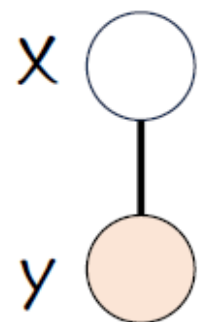
System of lin. Equations with D equations and D unknowns



Exercise: Linear Layer

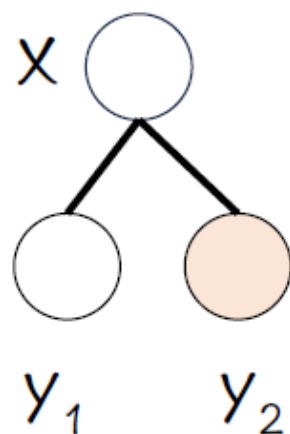
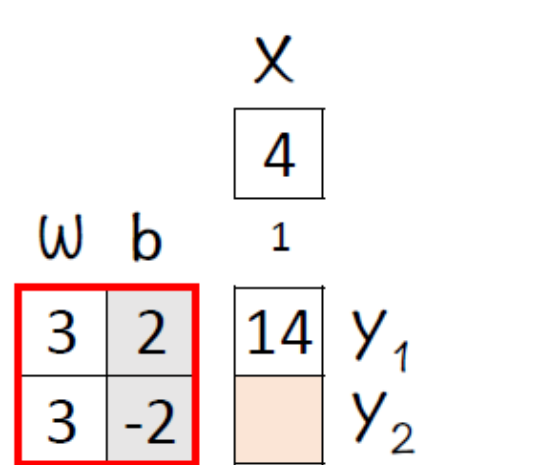


$$y = [w|b] \cdot [X|1] = wX + b$$



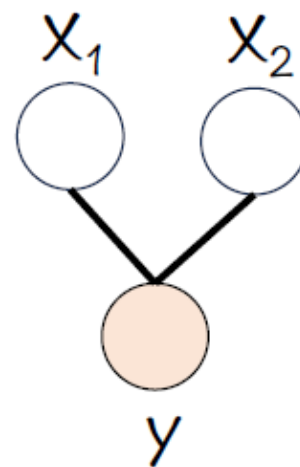
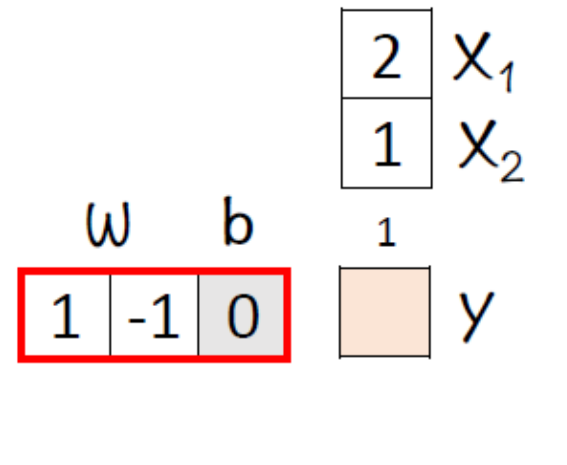
Exercise 1

12



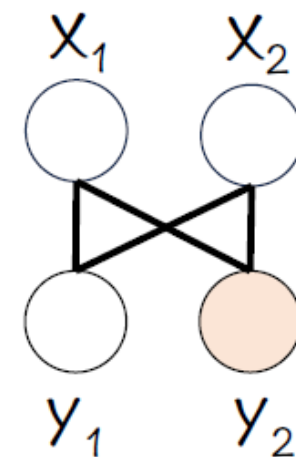
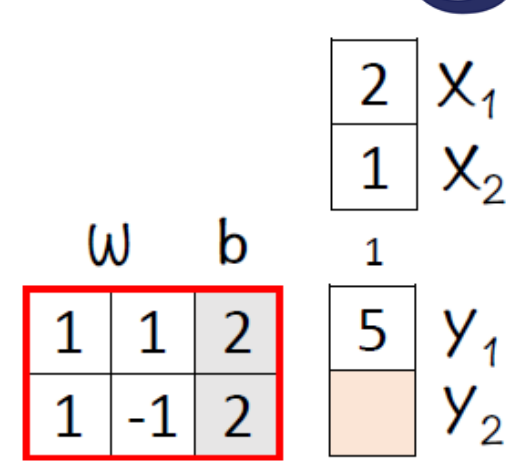
Exercise 2

10



Exercise 3

1



Exercise 4

3



Exercise: Linear Layer

ω			b	
1	1	0	1	
0	1	0	5	y_1
1	-1	0		y_2
			1	y_3

x_1

x_2

y_1

y_2

y_3

Exercise 5

2

ω				b	
1	1	0	0	1	
0	1	1	0	4	y_1
1	0	1	0		y_2
				5	y_3

x_1

x_2

x_3

y_1

y_2

y_3

Exercise 6

3

ω			b	
1	1	0	1	
2	1	0	3	y_1
	1	0	5	y_2
			7	y_3

x_1

x_2

y_1

y_2

y_3

Exercise 7

3

ω				b	
1	1	0	0	1	
1	1	0	-2	7	y_1
1	1	0		5	y_2
				3	y_3

x_1

x_2

x_3

y_1

y_2

y_3

Exercise 8

-4



Non-Linear Regression Model

Non-linear Regression Model

Definition and Formulation:

- **Non-linear Regression Model:** When the regression equation is in terms of r –degree, $r > 1$, then it is called **non-linear regression model**
- **Multiple Non-linear Regression Model:** When more than one independent variables are there, then it is called **multiple non-linear regression model**
 - It is alternatively termed as **polynomial regression model**.
- In general, it takes the form

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_r x^r + \epsilon$$

- The estimated response is obtained as

$$\hat{y} = b_0 + b_1 x + b_2 x^2 + \dots + b_r x^r$$

Solving for Polynomial Regression Model

Model formulation:

Given that $(x_i, y_i); i = 1, 2, \dots, n$ are n pairs of observations.

Each observations would satisfy the equations:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \dots + \beta_r x_i^r + \epsilon_i$$

and
$$\hat{y}_i = b_0 + b_1 x_i + b_2 x_i^2 + \dots + b_r x_i^r + e_i$$

where, r is the degree of polynomial

ϵ_i is the i^{th} random error

e_i is the i^{th} residual error

Note: The number of observations, n , must be at least as large as $r + 1$, the number of parameters to be estimated.

Polynomial to Multiple Linear Regression

Transformation to Linear Regression:

The polynomial model can be transformed into a general linear regression model setting $x_1 = x, x_2 = x^2, \dots, x_n = x^r$.

Thus, the equation assumes the form:

$$y_i = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_r x_r + \varepsilon_i$$

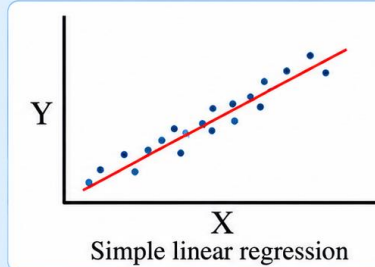
$$\hat{y}_i = b_0 + b_1 x_1 + b_2 x_2 + \dots + b_r x_r + e_i$$

This model then can be solved using the procedure followed for multiple linear regression model.

Linear versus Non-Linear Regression

Simple linear regression model:

$$Y = \beta_0 + \beta_1 x$$

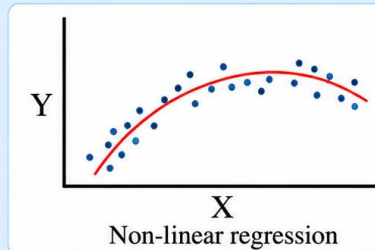


Issues:

- Too complex to solve. Many parameters, many variations!
- Usually, used advanced machine learning models, such as SVM, kNN, ANN, etc.

Simple non-linear regression model:

$$Y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_r x^r$$



Issues:

- Whether linear or non-linear model?
- If non-linear, then what is its degree $r \geq 2$?

Solution:

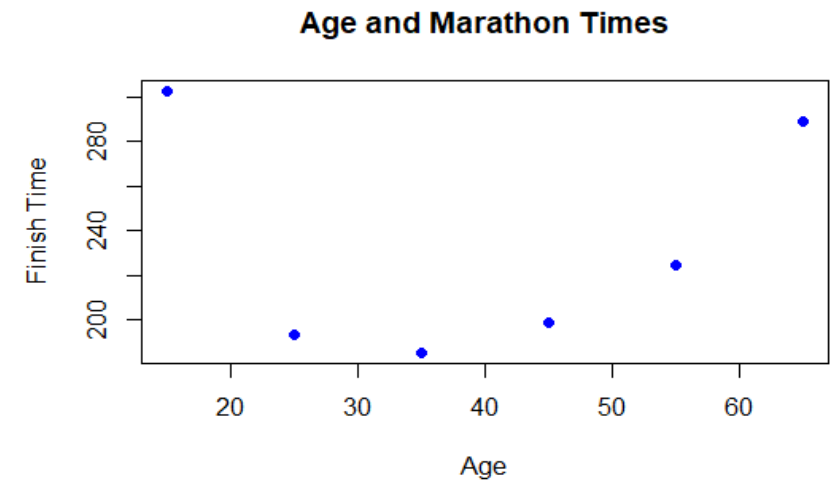
Take the R^2 measures for all models (with $r = 1, 2, \dots$) and then select that model with the higher value of R^2

X	Y
x_i	y_i

Nonlinear Regression: Marathon Data

- Example: The article “**Master’s Performance in the New York City Marathon**” (*British Journal of Sports Medicine* [2004]: 408–412) gave the following data on the average finishing time by age group for female participants in the New York City marathon.

Age Group	Representative Age	Average Finish Time
10-19	15	302.38
20-29	25	193.63
30-39	35	185.46
40-49	45	198.49
50-59	55	224.30
60-69	65	288.71

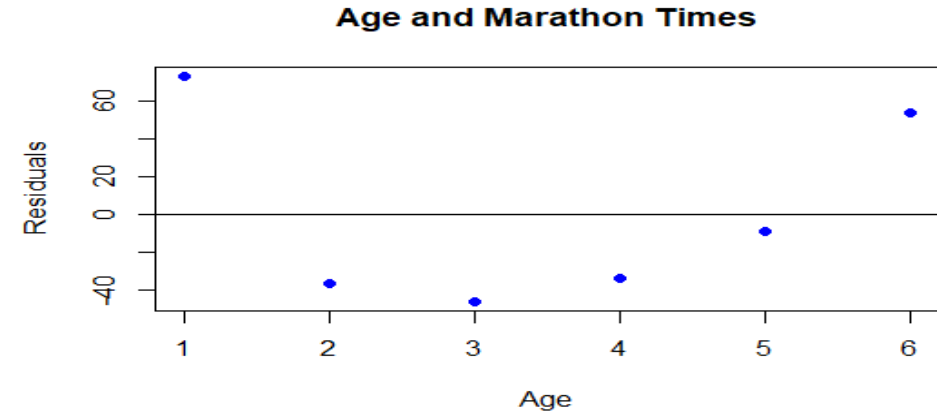


Correlation coefficient in Linear Case & Residual Plot

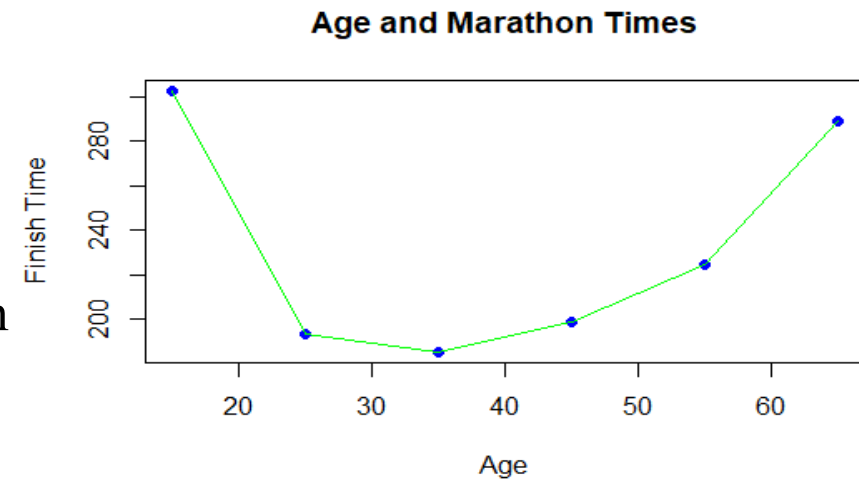
$$r = 0.03847689,$$

$$\text{Linear Regression line: } \hat{y} = 227.97 + 0.10x$$

Residual Plot:



- It is clear that no straight line can do a reasonable job of describing the relationship between x and y .
- However, the relationship can be described by a curve.
- Here the scatterplot looks like a parabola (the graph of a quadratic function).
- This suggests trying to find a quadratic function of the form $\hat{Y} = a + b_1X + b_2X^2$



Polynomial Regression

- Fit the regression line using the quadratic equation: $\hat{Y} = a + b_1X + b_2X^2$
- $y_i - (a + b_1x_i + b_2x_i^2)$ is called the error of estimate or residual for y_i .
- Principle of least square: determine a, b_1 and b_2 so that

$$E = \sum (y_i - a - b_1x_i - b_2x_i^2)^2, \text{ is minimum.}$$

$$\begin{aligned}\frac{\partial E}{\partial a} &= 0 = -2 \sum (y_i - a - b_1x_i - b_2x_i^2) \\ \frac{\partial E}{\partial b_1} &= 0 = -2 \sum (y_i - a - b_1x_i - b_2x_i^2)x_i \\ \frac{\partial E}{\partial b_2} &= 0 = -2 \sum (y_i - a - b_1x_i - b_2x_i^2)x_i^2\end{aligned}$$

- Hence,

$$\begin{aligned}\sum y_i &= na + b_1 \sum x_i + b_2 \sum x_i^2 \\ \sum x_i y_i &= a \sum x_i + b_1 \sum x_i^2 + b_2 \sum x_i^3 \\ \sum x_i^2 y_i &= a \sum x_i^2 + b_1 \sum x_i^3 + b_2 \sum x_i^4\end{aligned}$$

Example: Marathon Data

- By solving the above system of equation, we get

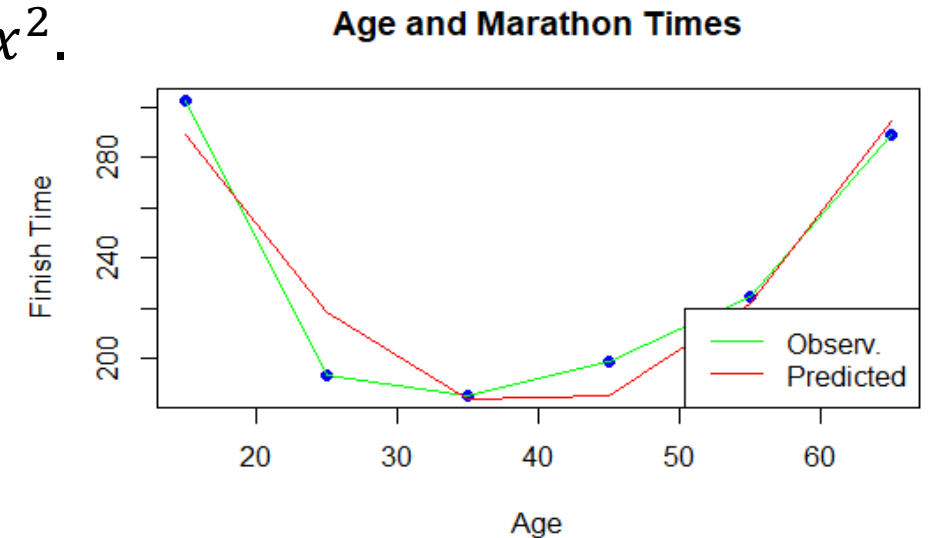
$$a = 462.0004453,$$

$$b_1 = -14.2054036$$

$$b_2 = 0.1788779$$

- Hence, the fitted line is : $\hat{y} = 462 - 14.2x + 0.179x^2$.
- The coefficient of Determination:

$$R_{XY}^2 = 1 - \frac{SS_{Resid}}{SS_{To}} = 0.9211164$$



Commonly used Linear Transformation

- An alternative to finding a curve to fit the data is to find a way to transform the x values and/or y values so that a scatterplot of the transformed data has a linear appearance.
- A **transformation** involves using a simple function of a variable in place of the variable itself.
- For example, instead of trying to describe the relationship between x and y , it might be easier to describe the relationship between $\sqrt{x}$ and y or between x and $\log(y)$.
- If we can describe the relationship between, say, $\sqrt{x}$ and y , we will still be able to predict the value of y for a given x value.

Equation	Transformation		Changed Equation
	Y'	X'	
$Y = \beta_0 x^{\beta_1}$	$Y' = \log y$	$x' = \log x$	$Y' = \log \beta_0 + \beta_1 x'$
$Y = \beta_0 e^{\beta_1 x}$	$Y' = \ln y$	$x' = x$	$Y' = \ln \beta_0 + \beta_1 x'$
$Y = \beta_0 + \beta_1 \log x$	$Y' = y$	$x' = \log x$	$Y' = \beta_0 + \beta_1 x'$
$Y = \frac{x}{\beta_0 x - \beta_1}$	$Y' = 1/y$	$x' = 1/x$	$Y' = \beta_0 - \beta_1 x'$

Note: Standardization of the data is needed when units are different and large-scale variable value difference in the data.

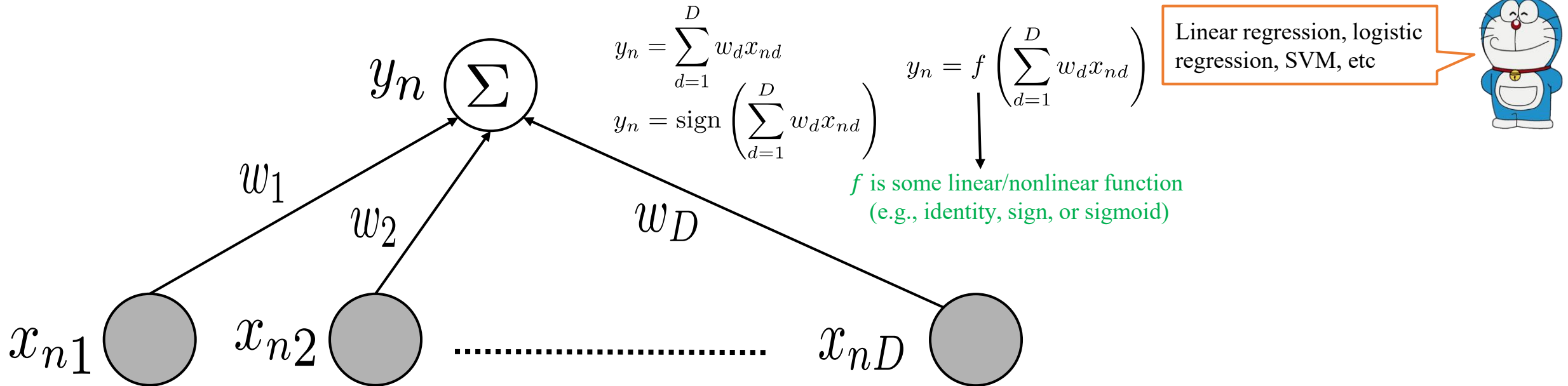


Artificial Neural Networks

Motivation for Neural Networks



- Linear models: Output produced by taking a linear combination of input features



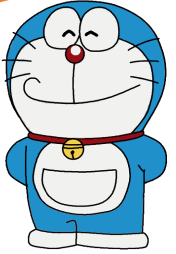
- A basic unit of the form $y = f(\mathbf{w}^T \mathbf{x})$ is known as the “Perceptron” (not to be confused with the Perceptron “algorithm”, which learns a linear classification model)
- This can’t however learn nonlinear functions or nonlinear decision boundaries

Although can kernelize to make them nonlinear

Loss Functions for Regression

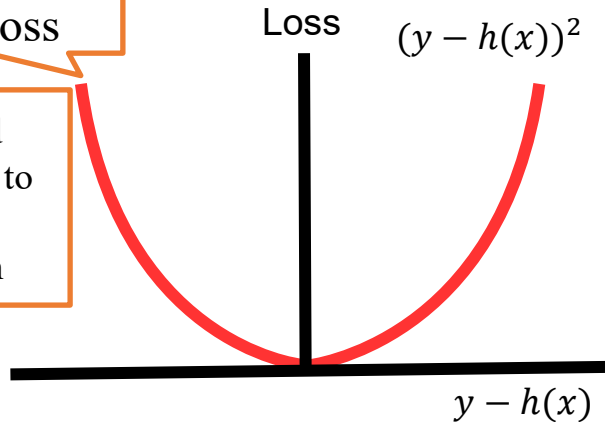
- Many possible loss functions for regression problems

Choice of loss function usually depends on the nature of the data. Also, some loss functions result in easier optimization problem than others



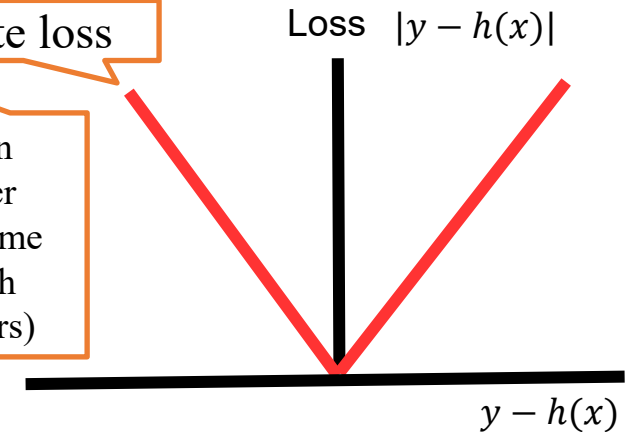
Squared loss

Very commonly used for regression. Leads to an easy-to-solve optimization problem



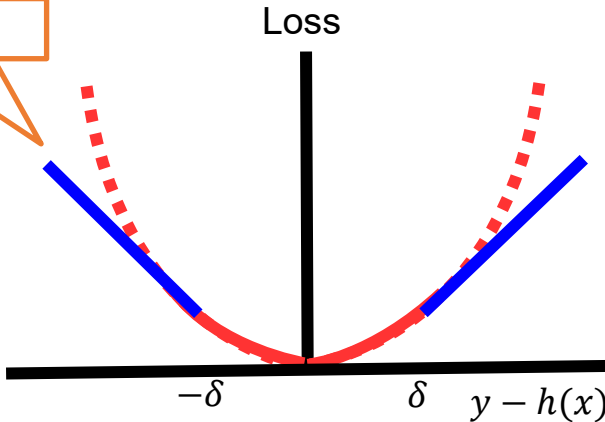
Absolute loss

Grows more slowly than squared loss. Thus better suited when data has some outliers (inputs on which model makes large errors)



Huber loss

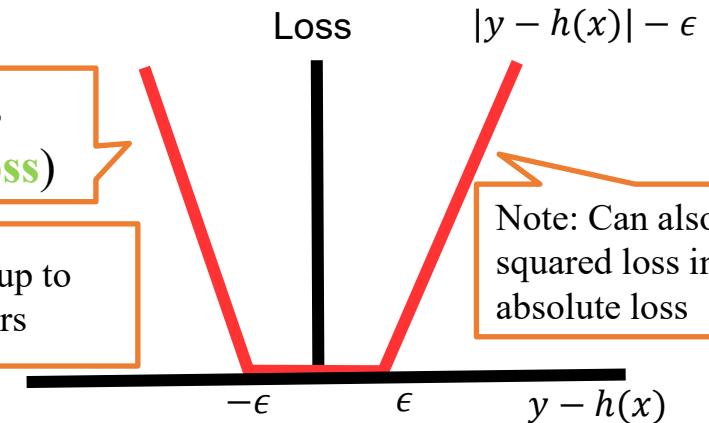
Squared loss for small errors (say up to δ); absolute loss for larger errors. Good for data with outliers



$$L_{\delta}(y, h(x)) = \begin{cases} \frac{1}{2} (y - h(x))^2 & \text{for } |y - h(x)| \leq \delta, \\ \delta |y - h(x)| - \frac{1}{2} \delta^2 & \text{otherwise.} \end{cases}$$

ϵ -insensitive loss (a.k.a. **Vapnik loss**)

Zero loss for small errors (say up to ϵ); absolute loss for larger errors

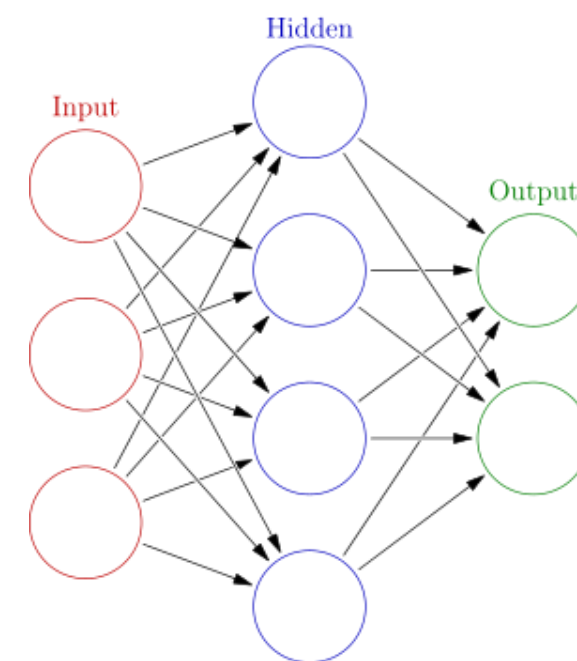
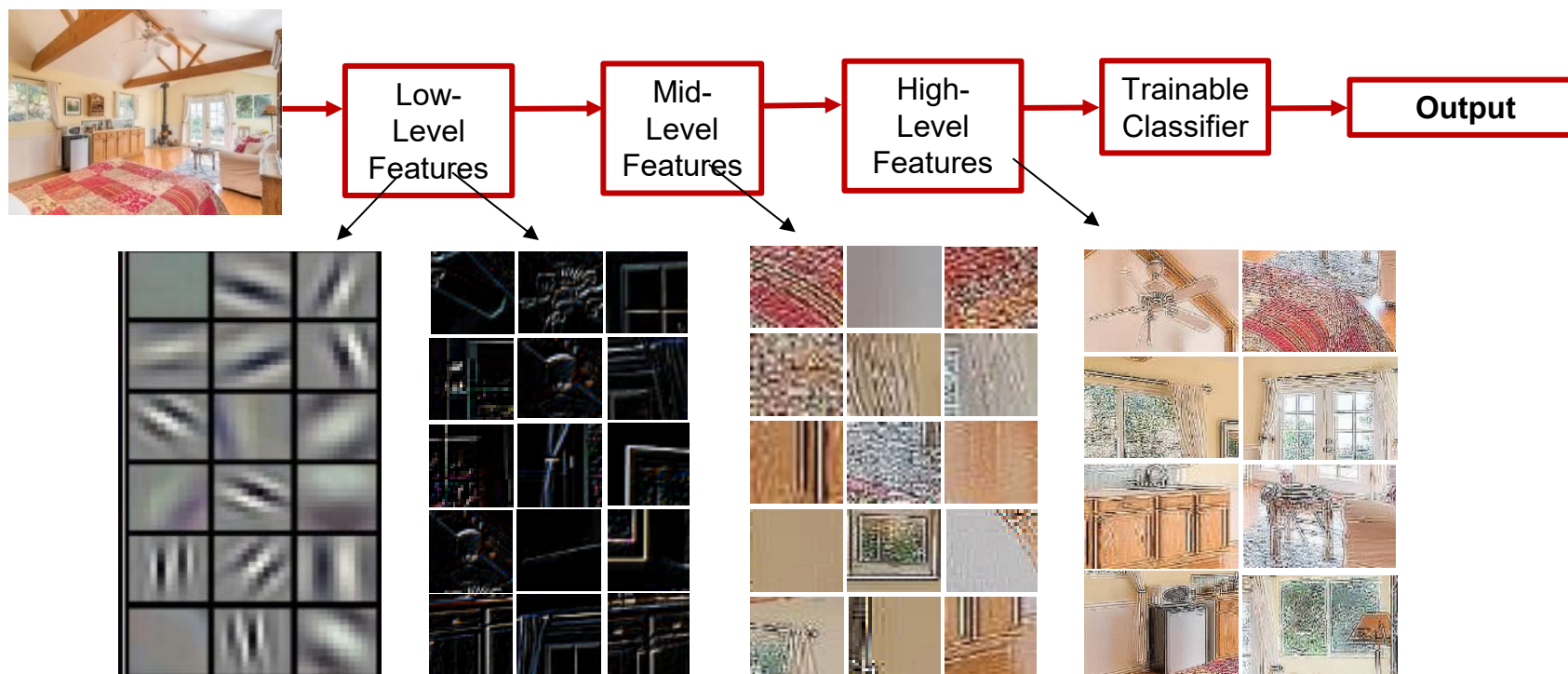


Note: Can also use squared loss instead of absolute loss

To read more about loss functions, see this [link](#)

Deep Learning

- DL applies a multi-layer process for learning rich hierarchical features (i.e., data representations)
 - Input image pixels → Edges → Textures → Parts → Objects



Neural Networks: Multi-layer Perceptron (MLP)



- An MLP is a network containing several Perceptron units across many layers
- An MLP consists of an **input layer**, an **output layer**, and **one or more hidden layers**

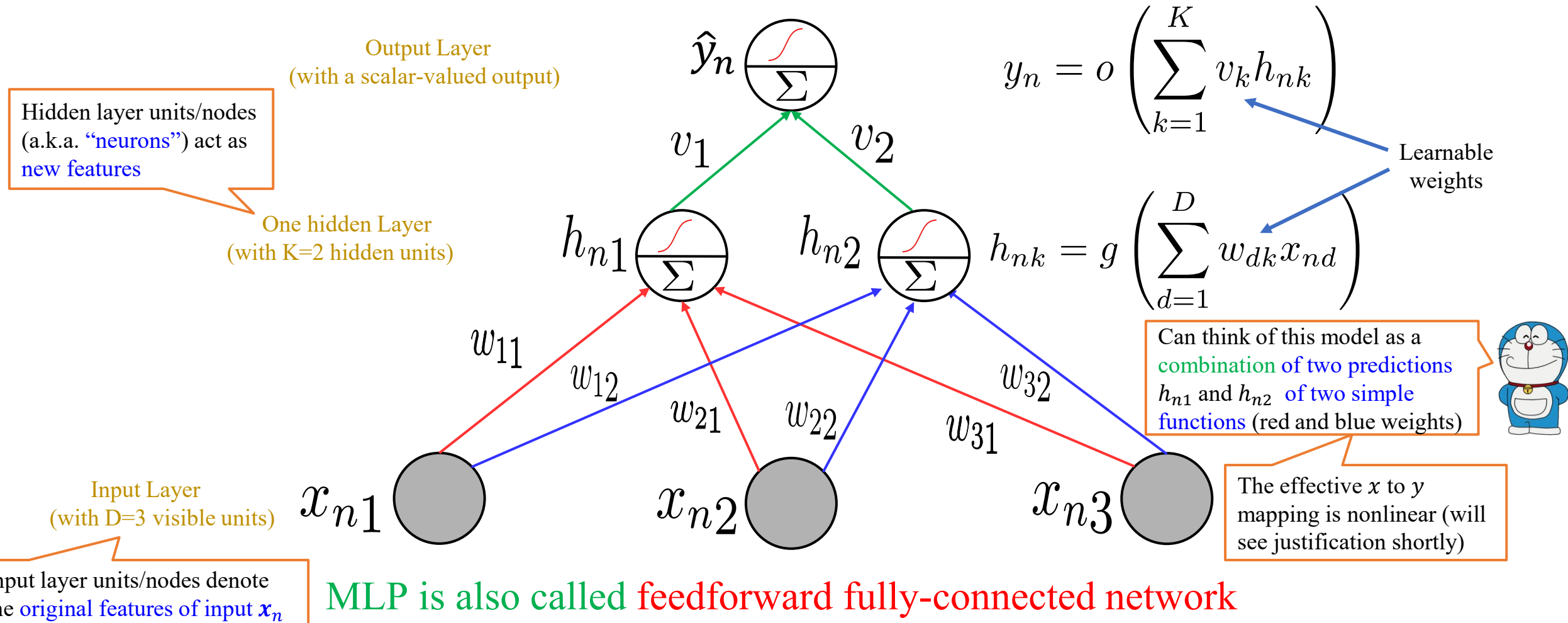


Illustration: Neural Net with Single Hidden Layer



- Compute K pre-activations for each input $\mathbf{x}_n$

A linear model with learnable weight vector $\mathbf{w}_k$

$$z_{nk} = \mathbf{w}_k^\top \mathbf{x}_n = \sum_{d=1}^D w_{dk} x_{nd} \quad (k = 1, 2, \dots, K)$$

- Apply nonlinear activation on each pre-act

Called a **hidden unit**

$$h_{nk} = g(z_{nk}) \quad (k = 1, 2, \dots, K)$$

- Apply a linear model with $\mathbf{h}_n$ acting as features

A linear model with learnable weight vector $\mathbf{v}$

Score of the input

$$s_n = \mathbf{v}^\top \mathbf{h}_n = \sum_{k=1}^K v_k h_{nk}$$

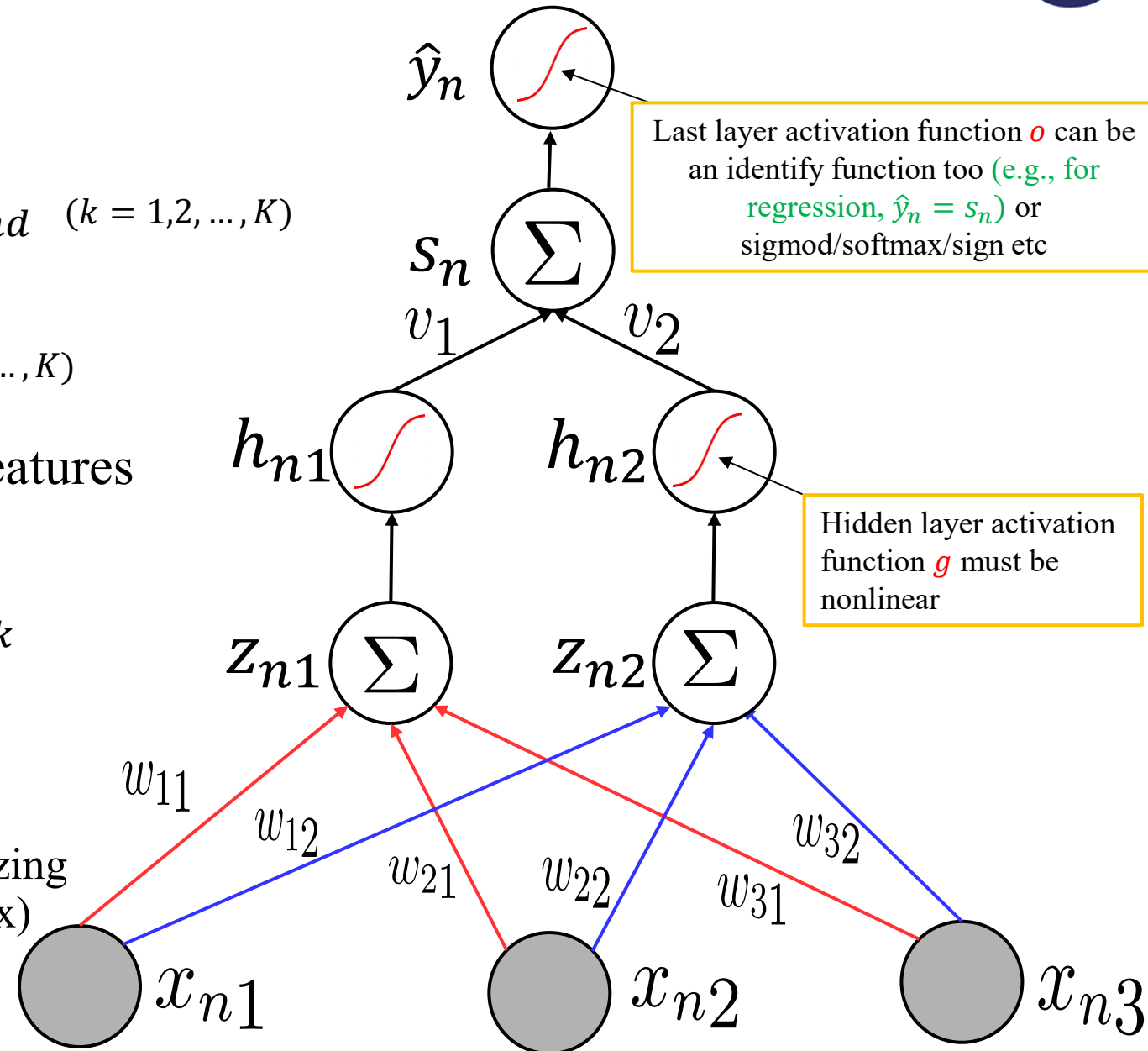
- Finally, output is produced as

Score converted to the actual prediction

$$\hat{y}_n = o(s_n)$$

- Unknowns $(\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_K, \mathbf{v})$ learned by minimizing some loss function (e.g., squared, logistic, softmax)

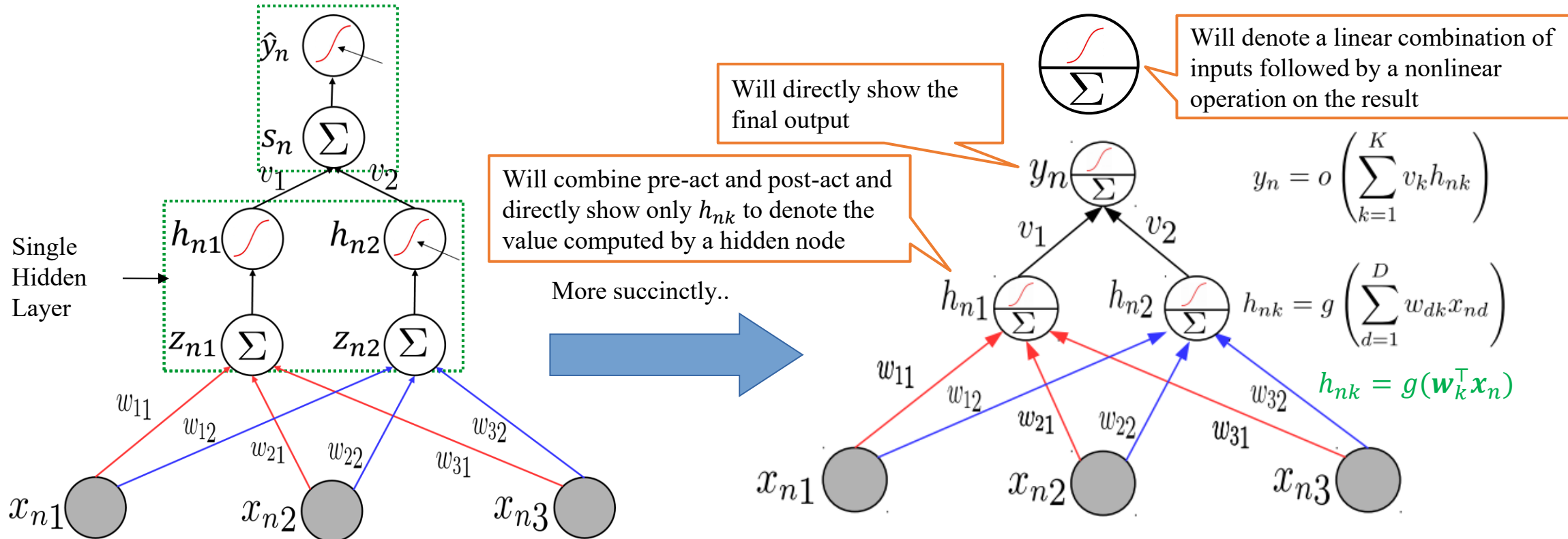
$$\text{Loss: } \mathcal{L}(\mathbf{W}, \mathbf{v}) = \sum_{n=1}^N \ell(y_n, \hat{y}_n)$$



Neural Nets: A Compact Illustration

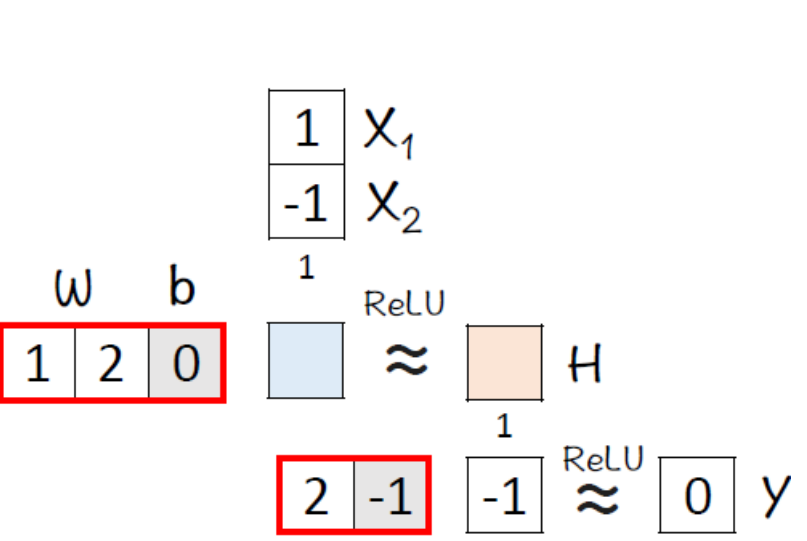


- Note: Hidden layer pre-act z_{nk} and post-act h_{nk} will be shown together for brevity
- Different layers may use different non-linear activations. Output layer may have none.



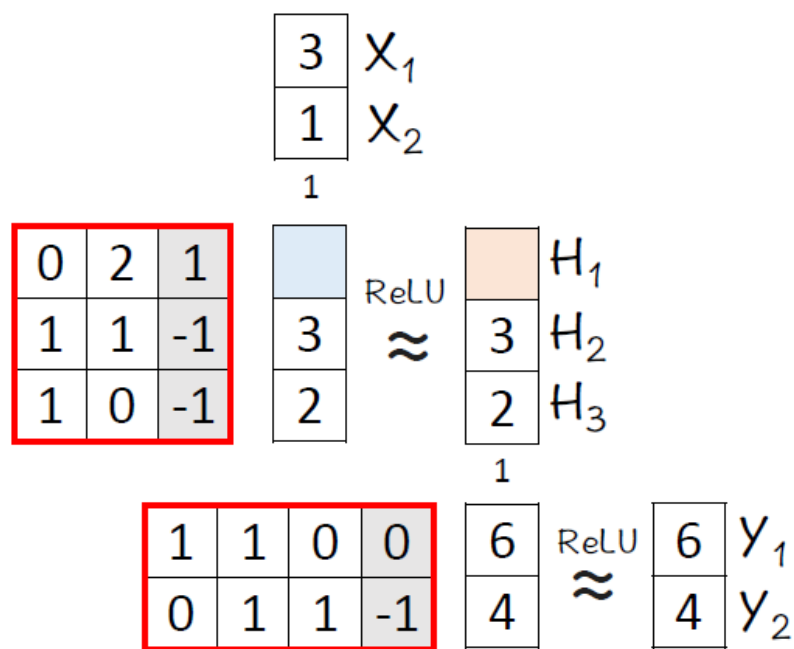
- Denoting $\mathbf{W} = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_K]$, $\mathbf{w}_k \in \mathbb{R}^D$, $\mathbf{h}_n = g(\mathbf{W}^\top \mathbf{x}_n) \in \mathbb{R}^K$ ($K = 2, D = 3$ above).
- **Note:** g applied elementwise on pre-activation vector $\mathbf{z}_n = \mathbf{W}^\top \mathbf{x}_n$

Exercise: Hidden Layer



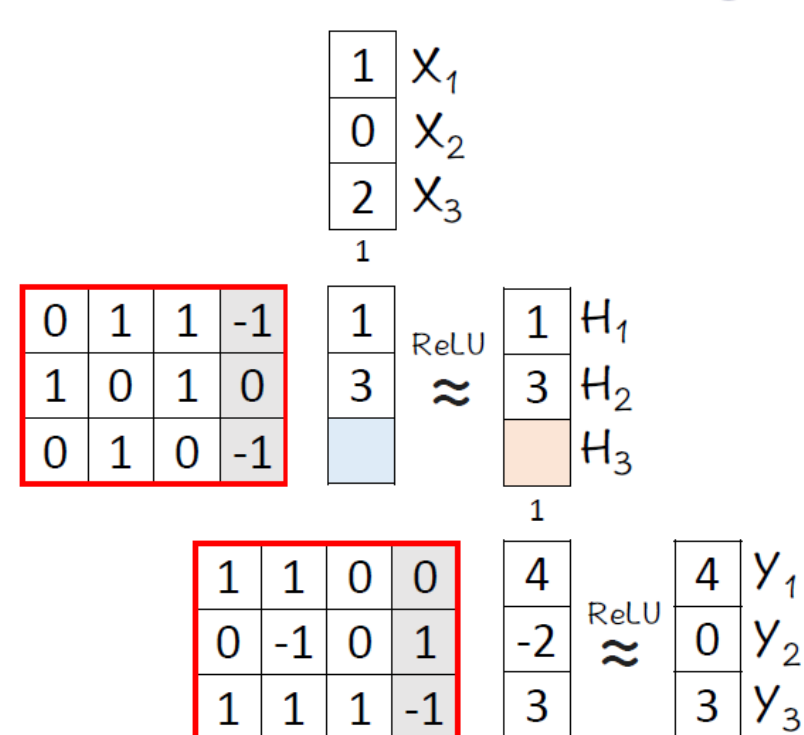
Exercise 1

-1	0
----	---



Exercise 2

3	3
---	---



Exercise 3

-1	0
----	---

Exercises: Hidden Layer

Exercise 4

Input: $X_1=2, X_2=1, X_3=3$

Weights: $\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & -1 \end{bmatrix}$

Biases: $\begin{bmatrix} 4 \\ 3 \\ 6 \\ 1 \end{bmatrix}$

ReLU: $\approx \begin{bmatrix} 4 \\ 3 \\ 6 \\ 1 \end{bmatrix}$

Hidden: $H_1=4, H_2=3, H_3=6, H_4=1$

Weights: $\begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & -2 \end{bmatrix}$

Biases: $\begin{bmatrix} 7 \\ \text{blue} \end{bmatrix}$

ReLU: $\approx \begin{bmatrix} 7 \\ \text{orange} \end{bmatrix}$

Output: $Y_1=7, Y_2=\text{orange}$

Diagram: A neural network with 3 input nodes (X_1, X_2, X_3), 4 hidden nodes (H_1, H_2, H_3, H_4), and 2 output nodes (Y_1, Y_2). All nodes are fully connected.

Weights: $\begin{bmatrix} 3 & 3 \end{bmatrix}$

Exercise 5

Input: $X_1=2, X_2=-1$

Weights: $\begin{bmatrix} 0 & 0 & -1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ -1 & 0 & 3 \\ 1 & 1 & 0 \end{bmatrix}$

Biases: $\begin{bmatrix} -1 \\ 3 \\ 0 \\ 1 \\ 1 \end{bmatrix}$

ReLU: $\approx \begin{bmatrix} 0 \\ 3 \\ 0 \\ 1 \\ 1 \end{bmatrix}$

Hidden: $H_1=0, H_2=3, H_3=0, H_4=1, H_5=1$

Weights: $\begin{bmatrix} 1 & 1 & 1 & -1 & -1 & 1 \end{bmatrix}$

Biases: $\begin{bmatrix} \text{blue} \end{bmatrix}$

ReLU: $\approx \begin{bmatrix} \text{orange} \end{bmatrix}$

Output: $Y=\text{orange}$

Diagram: A neural network with 2 input nodes (X_1, X_2), 5 hidden nodes (H_1, H_2, H_3, H_4, H_5), and 1 output node (Y). All nodes are fully connected.

Weights: $\begin{bmatrix} 2 & 2 \end{bmatrix}$

Exercise 6

Input: $X_1=3, X_2=2$

Weights: $\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 2 \\ 0 & -1 & 1 \\ 1 & -1 & 0 \end{bmatrix}$

Biases: $\begin{bmatrix} 2 \\ 3 \\ -1 \\ 2 \\ \text{blue} \\ 1 \end{bmatrix}$

ReLU: $\approx \begin{bmatrix} 2 \\ 3 \\ 0 \\ 2 \\ \text{orange} \\ 1 \end{bmatrix}$

Hidden: $H_1=2, H_2=3, H_3=0, H_4=2, H_5=\text{orange}, H_6=1$

Weights: $\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 0 \end{bmatrix}$

Biases: $\begin{bmatrix} 8 \end{bmatrix}$

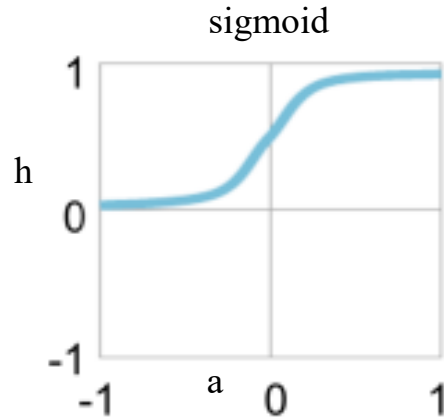
ReLU: $\approx \begin{bmatrix} 8 \end{bmatrix}$

Output: $Y=8$

Diagram: A neural network with 2 input nodes (X_1, X_2), 6 hidden nodes ($H_1, H_2, H_3, H_4, H_5, H_6$), and 1 output node (Y). All nodes are fully connected.

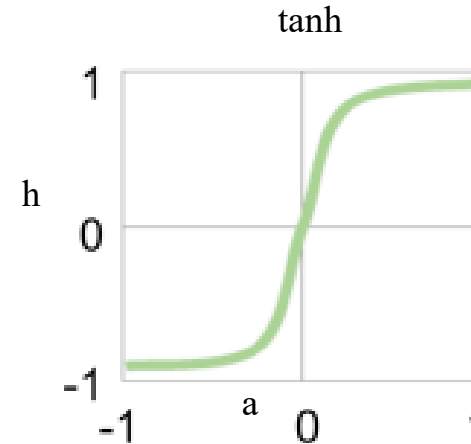
Weights: $\begin{bmatrix} -1 & 0 \end{bmatrix}$

Activation Functions: Some Common Choices



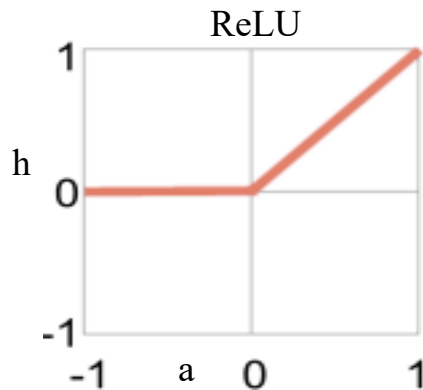
For sigmoid as well as tanh, gradients saturate (become close to zero as the function tends to its extreme values)

Sigmoid: $h = \sigma(a) = \frac{1}{1 + \exp(-a)}$



Preferred more than sigmoid. Helps keep the mean of the next layer's inputs close to zero (with sigmoid, it is close to 0.5)

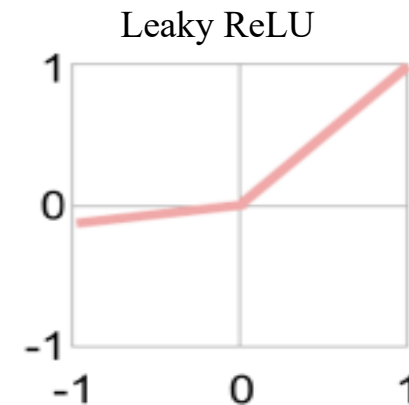
tanh (tan hyperbolic): $h = \frac{\exp(a) - \exp(-a)}{\exp(a) + \exp(-a)} = 2\sigma(2a) - 1$



ReLU and Leaky ReLU are among the most popular ones (also efficient to compute)

Helps fix the dead neuron problem of ReLU when a is a negative number

ReLU (Rectified Linear Unit): $h = \max(0, a)$



$$y = \mathbf{v}^\top (g(\mathbf{W}^\top \mathbf{x})) = \mathbf{v}^\top \mathbf{W}^\top \mathbf{x}$$

Still linear

Without nonlinear activation, a deep neural network is equivalent to a linear model no matter how many layers we use

Leaky ReLU: $h = \max(\beta a, a)$
where β is a small positive number

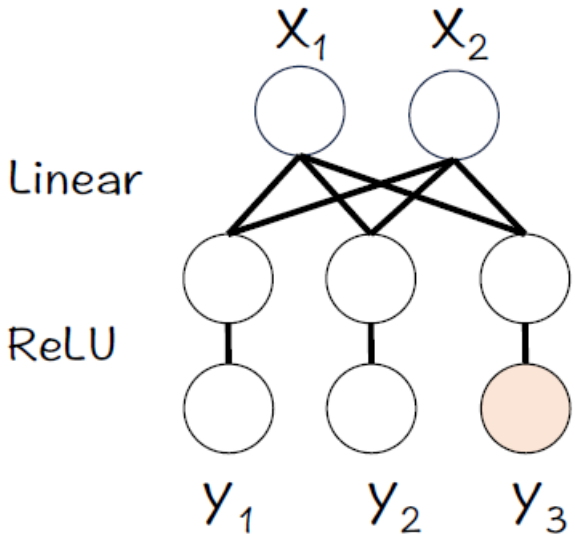
Most activation functions are monotonic but there exist some non-monotonic activation functions as well (e.g., Swish: $a \times \sigma(\beta a)$)



Exercises: Activation Functions (ReLU, Sigmoid, Tanh)



w			b		x_1	x_2	
1	-1	0	1	ReLU	1	y_1	
1	-2	0	-1	$\approx$	0	y_2	
1	-3	0	-3			y_3	



Exercise 1

0

w			b		x_1	x_2	
1	-1	0	1	Sigmoid	0.5	y_1	
1	-2	0	-1	$\sigma_{\{3\}}$		y_2	
1	-3	0	-3	$\approx$	0	y_3	

3 Level Quantization
for hand calculation

Integer	Sigmoid
≥ 2	1
-1, 0, 1	0.5
≤ -2	0

Exercise 2

0.5

w			b		x_1	x_2	
1	-1	-2	-1	Tanh	0	y_1	
1	-2	-2	-3	$\{3\}$		y_2	
1	-3	-2	-5	$\approx$	-1	y_3	

3 Level Quantization
for hand calculation

Integer	Sigmoid	Tanh
≥ 2	1	1
-1, 0, 1	0.5	0
≤ -2	0	-1

Exercise 3

-1

MLP Can Learn Any Nonlinear Function

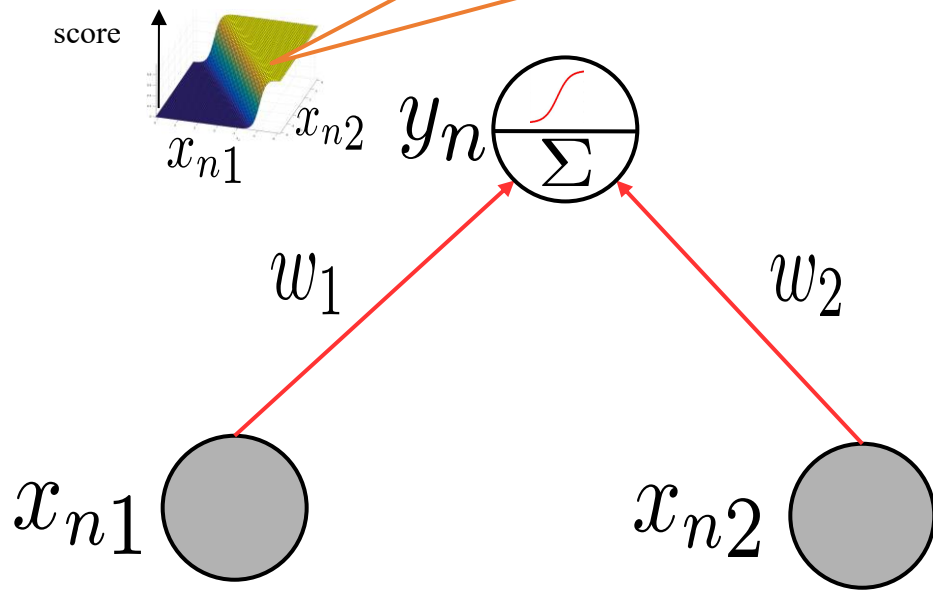


- An MLP can be seen as a composition of **multiple linear models combined nonlinearly**

Score monotonically increases. One-sided increase (not ideal for learning nonlinear decision boundaries). **Just a linear model**

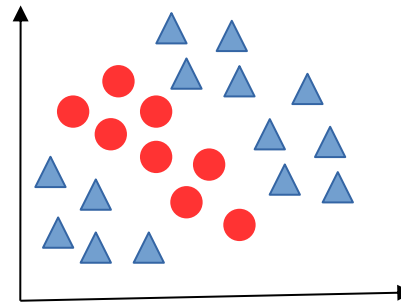
Obtained by composing the two one-sided increasing score functions (using learnable weights v_1 and v_2 to “add” them). This can now learn the nonlinear decision boundary (high score in the middle)

High-score in the middle and low-score on either of the two sides of it. Exactly what we want for the given classification problem

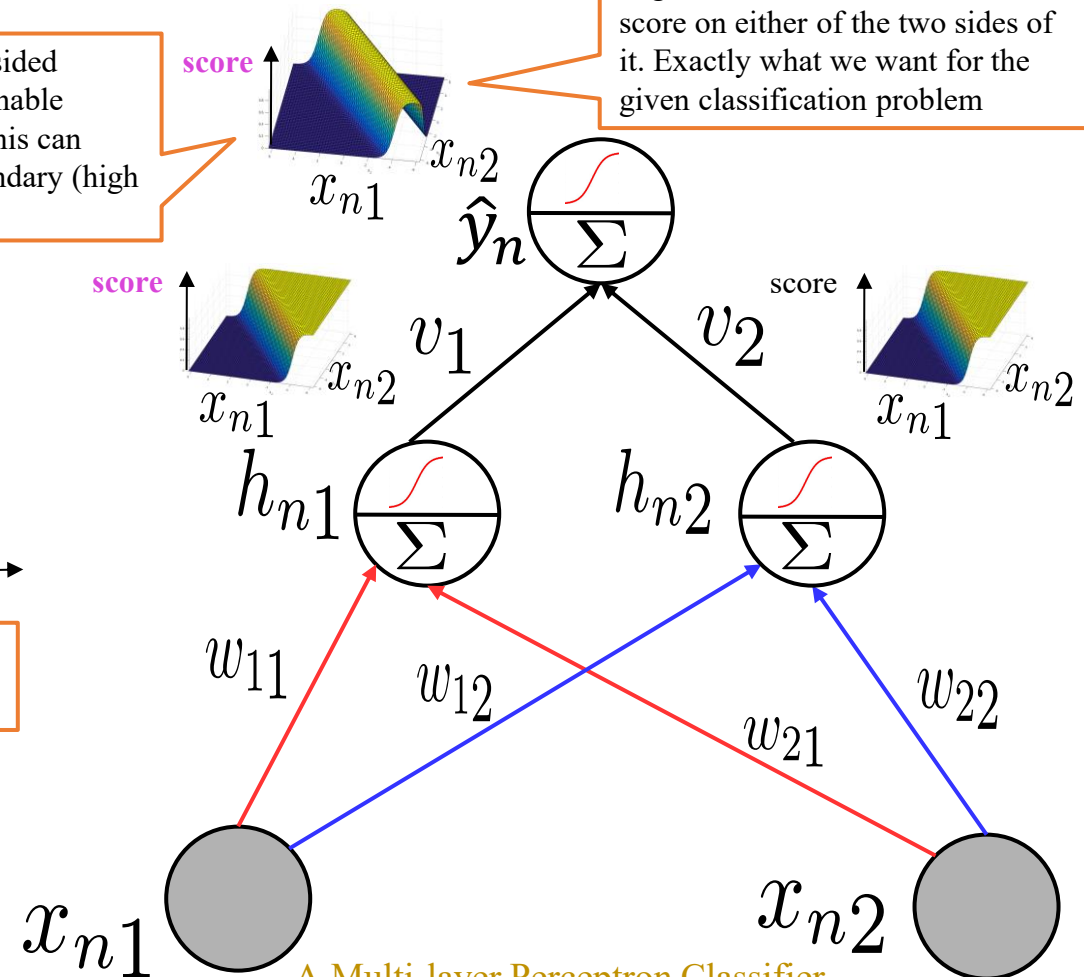


Standard Single “Perceptron” Classifier (no hidden units)

A single hidden layer MLP with sufficiently large number of hidden units can approximate any function ([Hornik, 1991](#))



A nonlinear classification problem



A Multi-layer Perceptron Classifier
(one hidden layer with 2 units)
Capable of learning nonlinear boundaries

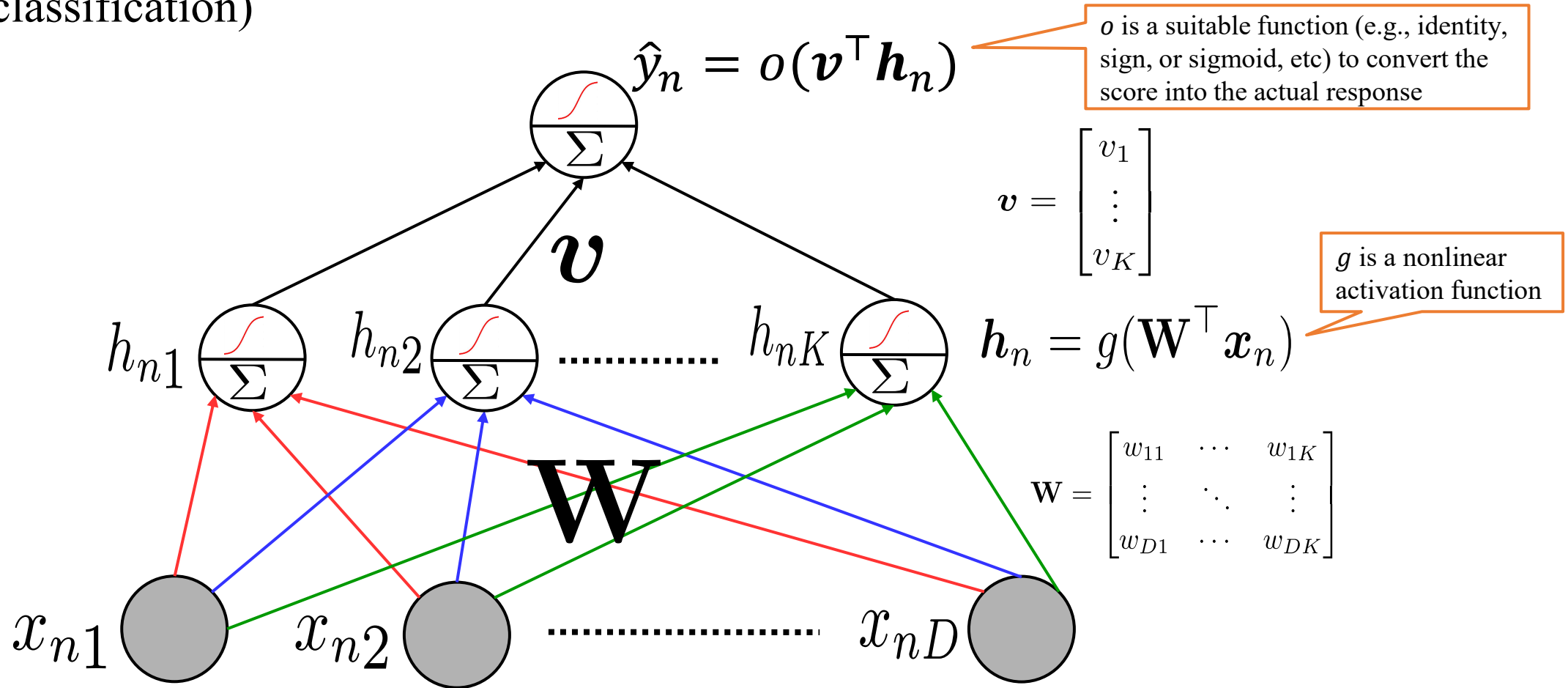




Examples of some basic NN/MLP architectures

Single Hidden Layer and Single Output

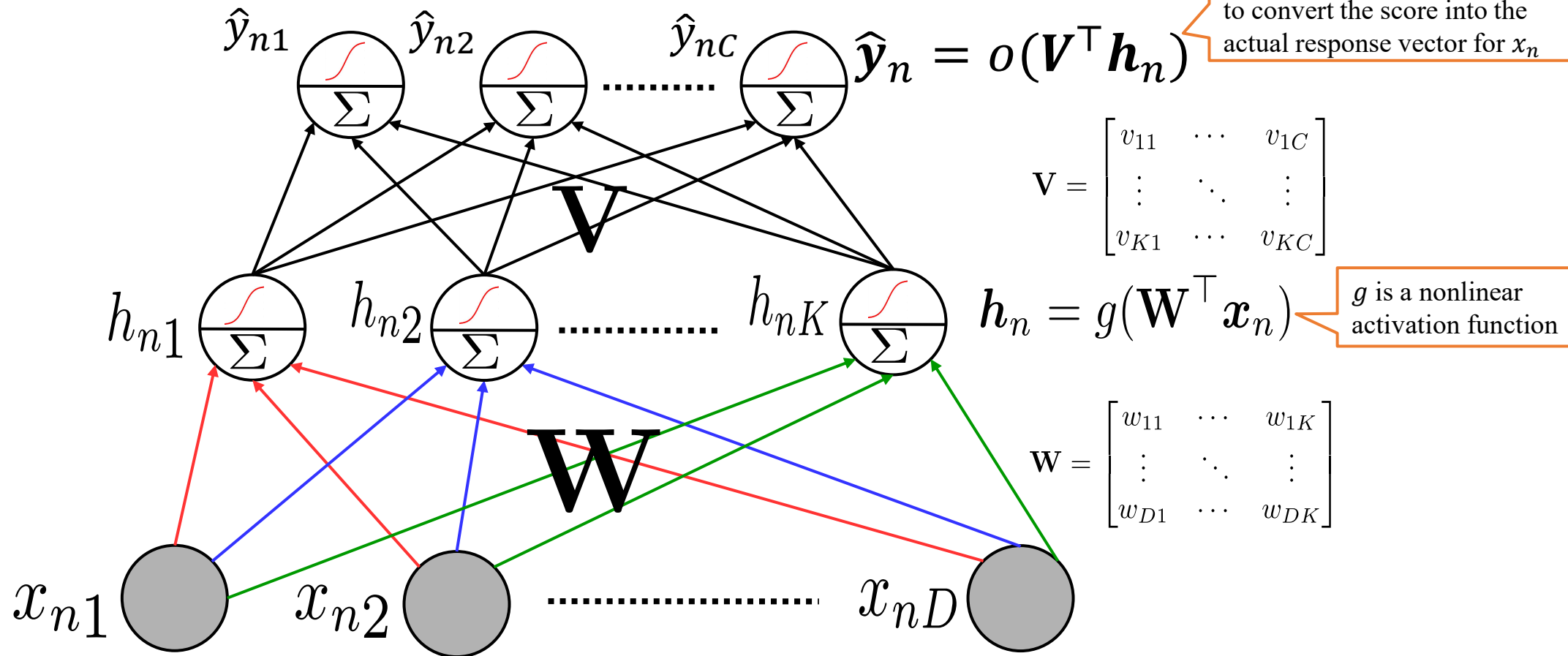
- One hidden layer with K nodes and a single output (e.g., scalar-valued regression or binary classification)



Single Hidden Layer and Multiple Outputs



- One hidden layer with K nodes and a vector of C outputs (e.g., vector-valued regression or multi-class classification or multi-label classification)

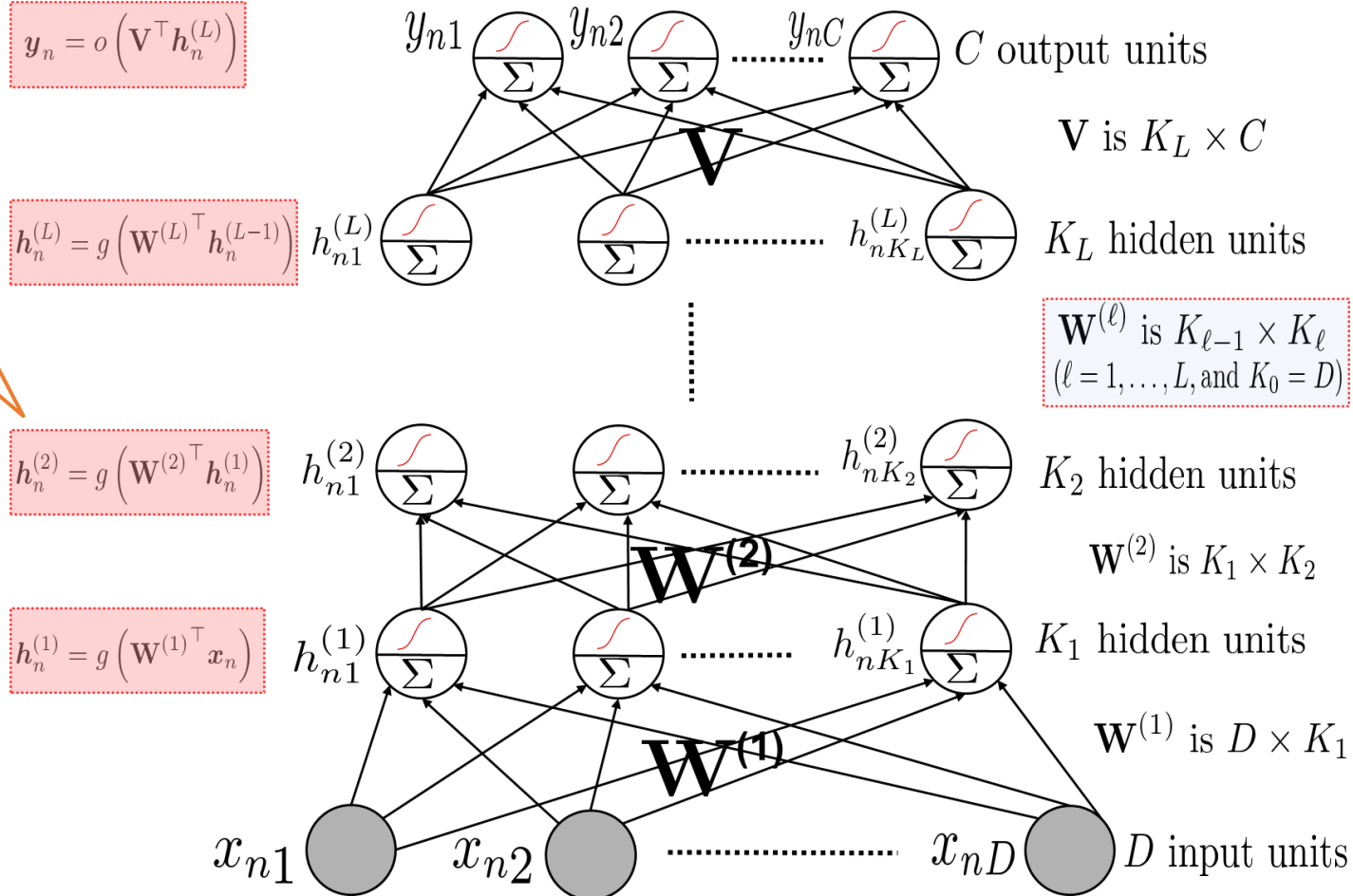


Multiple Hidden Layers (One/Multiple Outputs)

- Most general case: Multiple hidden layers with (with same or different number of hidden nodes in each) and a scalar or vector-valued output

Each hidden layer uses a nonlinear activation function g (essential, otherwise the network can't learn nonlinear functions and reduces to a linear model)

Note: Nonlinearity g is applied **element-wise** on its inputs so $h_n^{(\ell)}$ has the same size as vector $W^{(\ell)} h_n^{(\ell-1)}$



The Bias Term

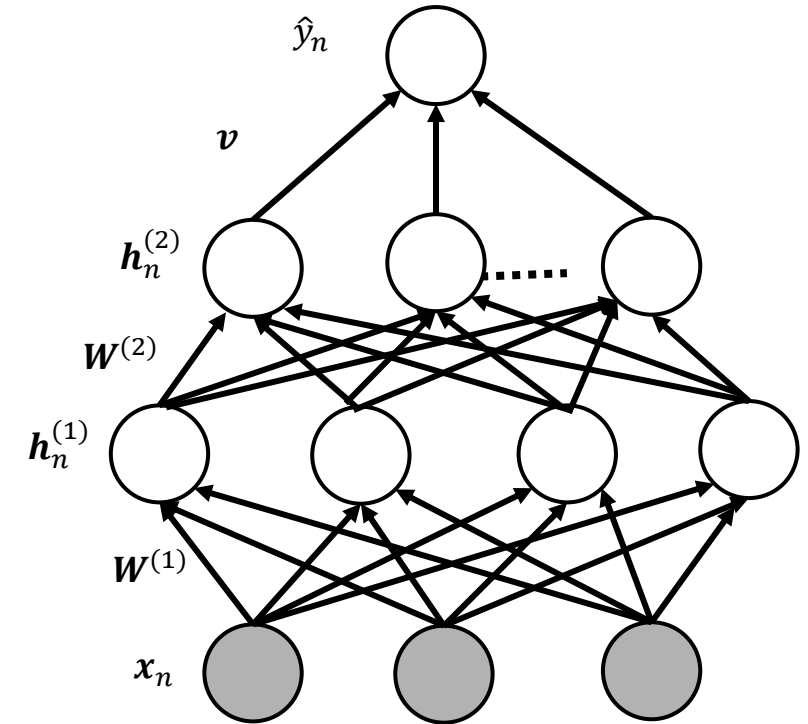


- Each layer's pre-activations $\mathbf{z}_n^{(\ell)}$ have an add bias term $\mathbf{b}^{(\ell)}$ (has the same size as $\mathbf{z}_n^{(\ell)}$ and $\mathbf{h}_n^{(\ell)}$) as well

$$\mathbf{z}_n^{(\ell)} = \mathbf{W}^{(\ell)\top} \mathbf{x}_n^{(\ell-1)} + \mathbf{b}^{(\ell)}$$

$$\mathbf{h}_n^{(\ell)} = g(\mathbf{z}_n^{(\ell)})$$

- Bias term increases the expressiveness of the network and ensures that we have nonzero activations/pre-activations even if this layer's input is a vector of all zeros
- Note that the bias term is the same for all inputs (does not depend on n)
- The bias term $\mathbf{b}^{(\ell)}$ is also learnable



Exercises: Artificial Neuron

w			b		
2	1	-5			

2	x_1
-1	x_2

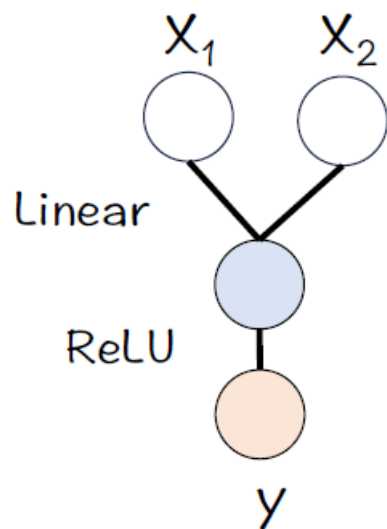
1

 ReLU

 $\approx$

--

 y



Exercise 1

-2
0

w				b		
1	1	0	0			
1	0	-1	0			
1	-1	0	0			

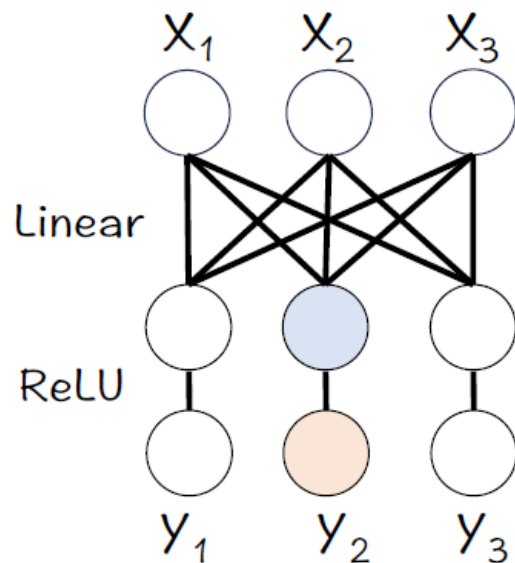
2	x_1
3	x_2
4	x_3

5
-1

 ReLU

 $\approx$

3	y_1
	y_2
0	y_3



Exercise 2

-2
0

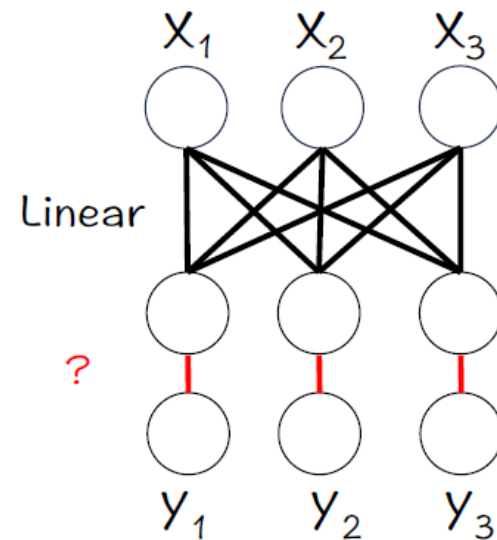
w				b		
1	-1	0	0			
1	0	-1	0			
1	0	0	0			

2	x_1
4	x_2
3	x_3

-2
-1
2

 $\approx$

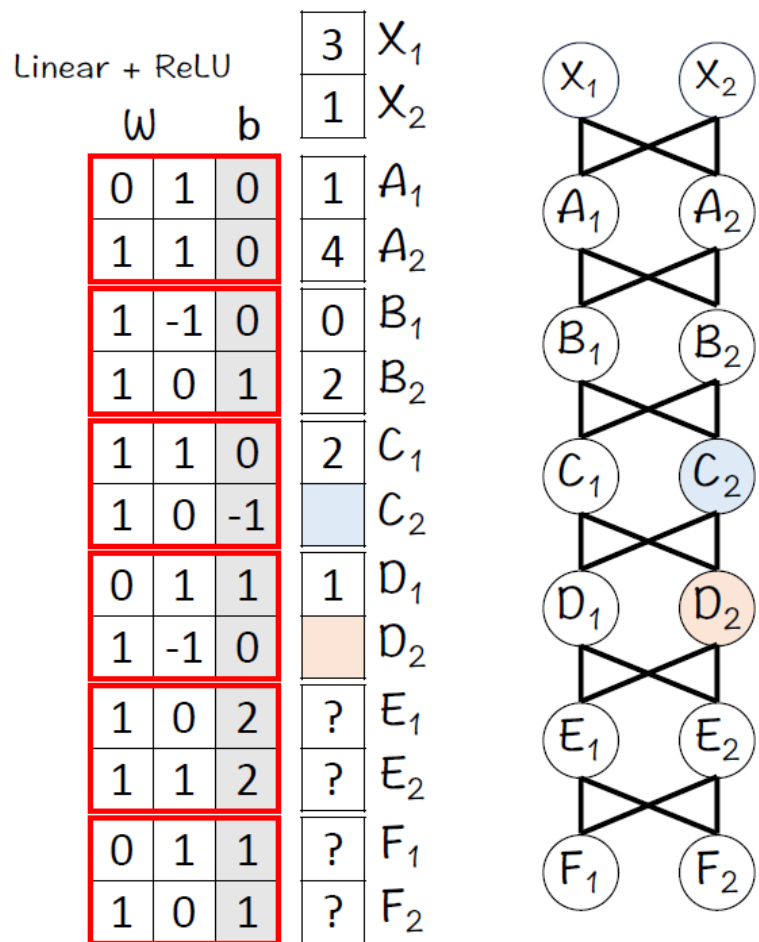
?	y_1
0.5	y_2
1	y_3



Exercise 3

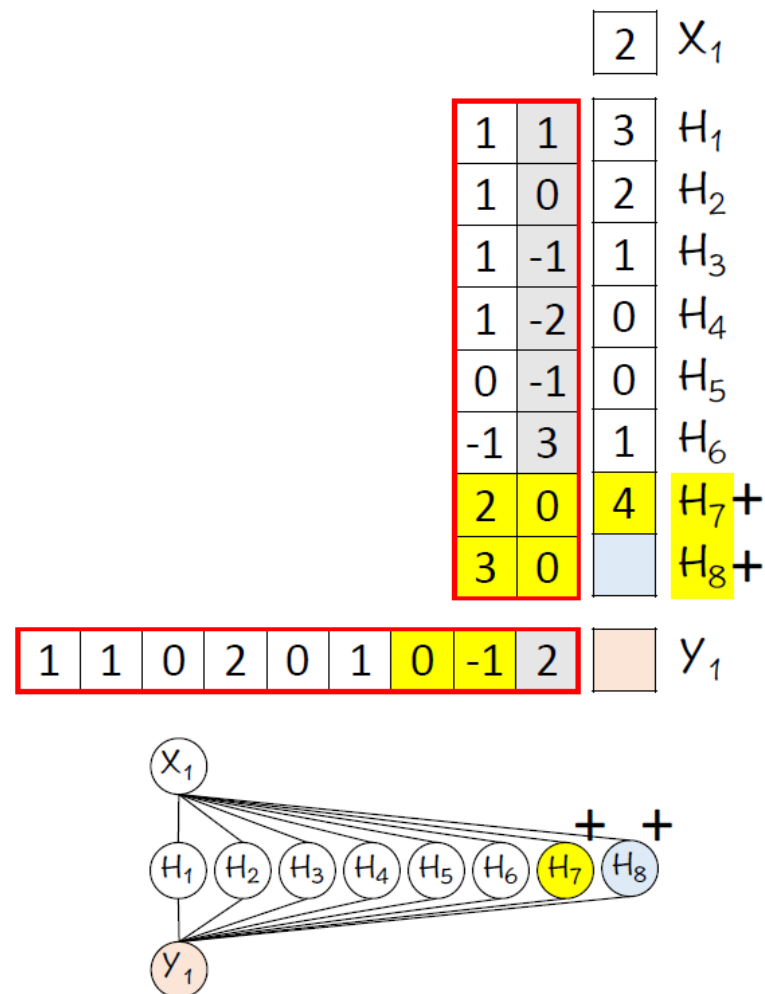
Sigmoid

Exercises: Deep and Wide



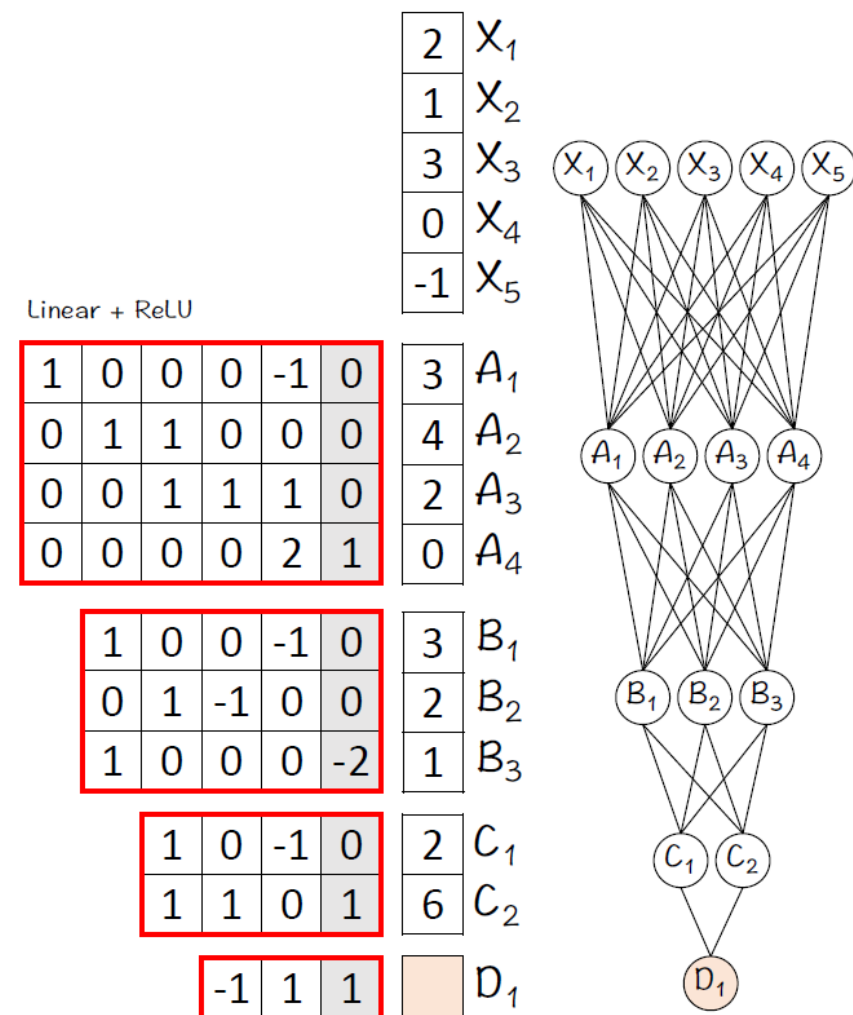
Exercise 1

0
2



Exercise 2

6
2

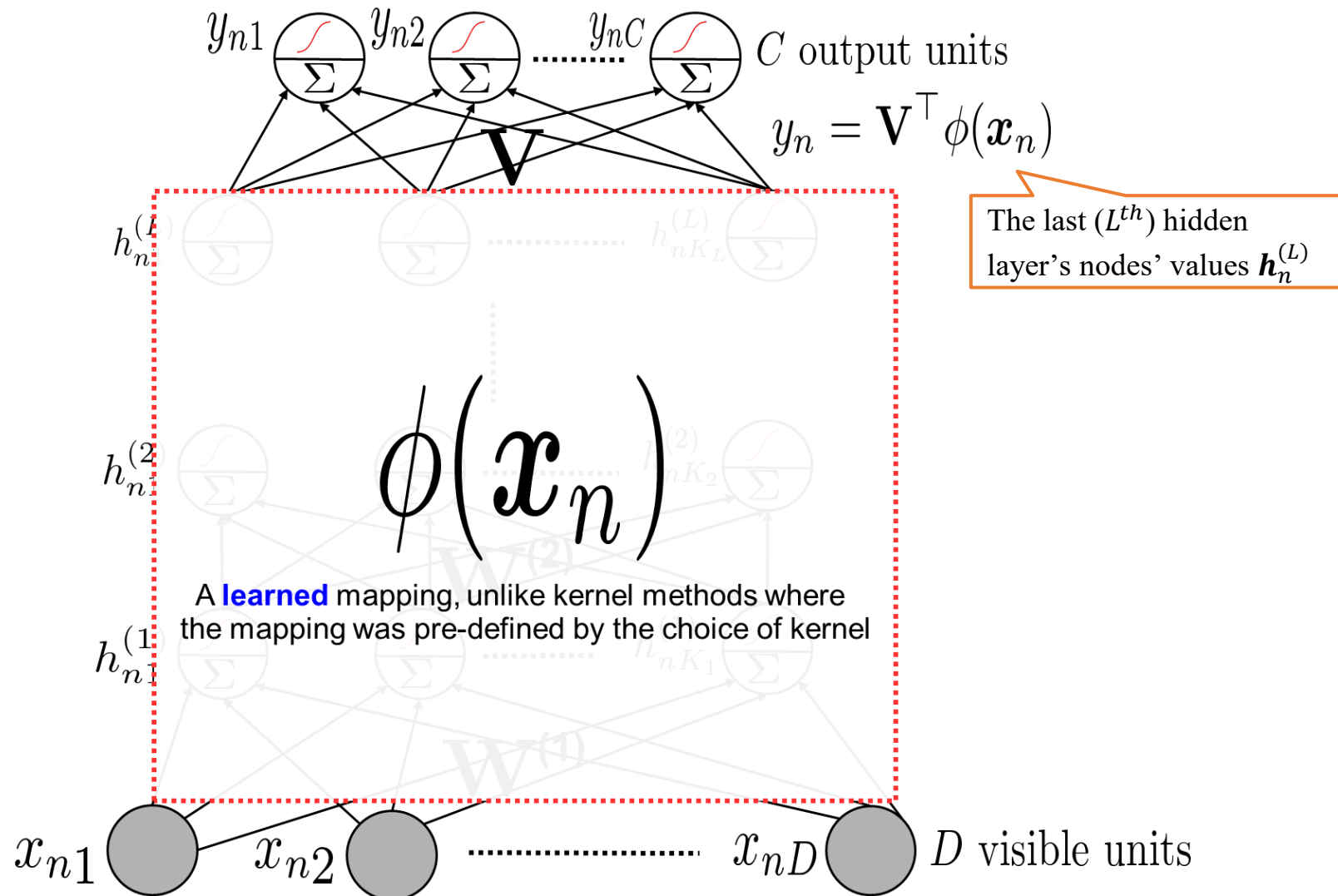


Exercise 3

5

Comments: Neural Nets are Feature Learners

- Hidden layers can be seen as learning a feature rep. $\phi(\mathbf{x}_n)$ for each input $\mathbf{x}_n$

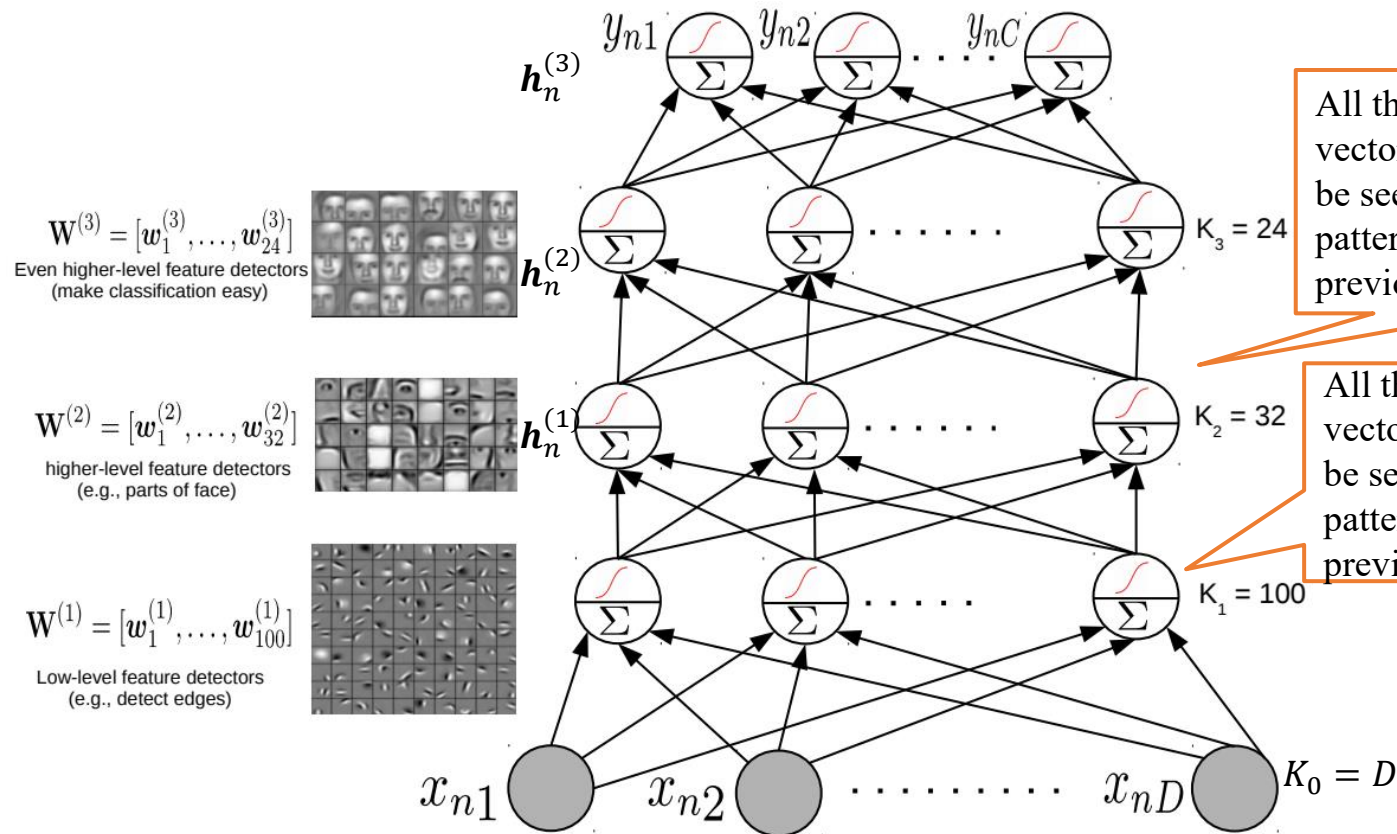


Features Learned by a NNs

- Node values in each hidden layer tell us how much a “learned” feature is active in $\mathbf{x}_n$
- Hidden layer weights are like pattern/feature-detector/filter
- $\mathbf{w}_k^{(\ell)} \in \mathbb{R}^{K_{\ell-1}}$ denotes “feature detector” for feature k of layer ℓ
- For input $\mathbf{x}_n$, $h_{nk}^{(\ell)} = g(\mathbf{w}_k^{(\ell)\top} \mathbf{h}_n^{(\ell-1)})$ is the value of feature k in layer ℓ

$\mathbf{W}^{(\ell)} = [\mathbf{w}_1^{(\ell)}, \mathbf{w}_2^{(\ell)}, \dots, \mathbf{w}_{K_\ell}^{(\ell)}]$ is $K_{\ell-1} \times K_\ell$ matrix of weights between layer $\ell - 1$ and ℓ

$\mathbf{W}^{(\ell)}$ collectively represents the K_ℓ feature detectors for layer ℓ



All the incoming weights (a vector) to a hidden node can be seen as representing a pattern/feature-detector of previous layer's inputs

E.g., $\mathbf{w}_{32}^{(2)}$ denotes a feature detector

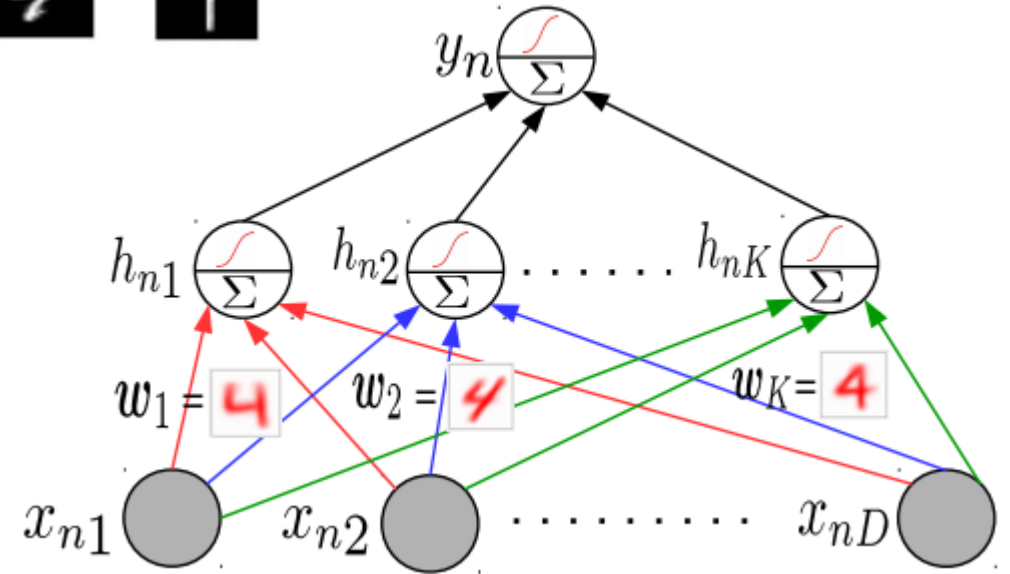
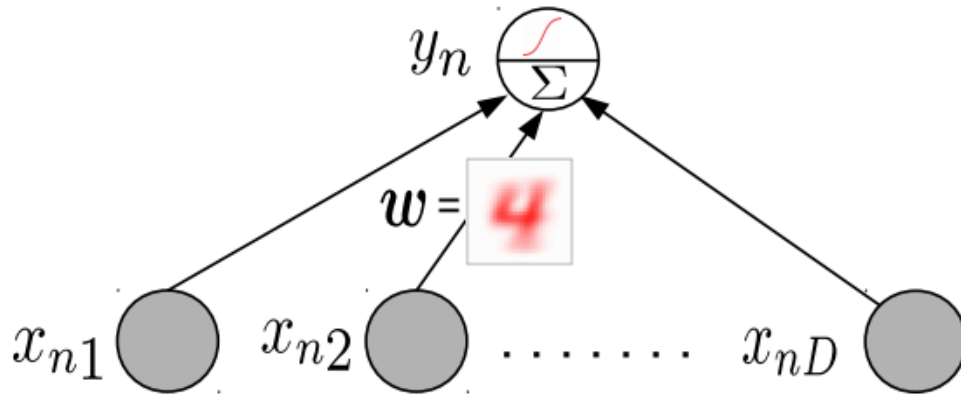
All the incoming weights (a vector) to a hidden node can be seen as representing a pattern/feature-detector of previous layer's inputs

E.g., $\mathbf{w}_{100}^{(1)}$ denotes a feature detector

Why Neural Networks Work Better: Another View



- Linear models tend to only learn the “average” pattern
 - E.g., Weight vector of a linear classification model represent average pattern of a class
- Deep models can learn multiple patterns (each hidden node can learn one pattern)
 - Thus deep models can learn to capture more subtle variations that a simpler linear model

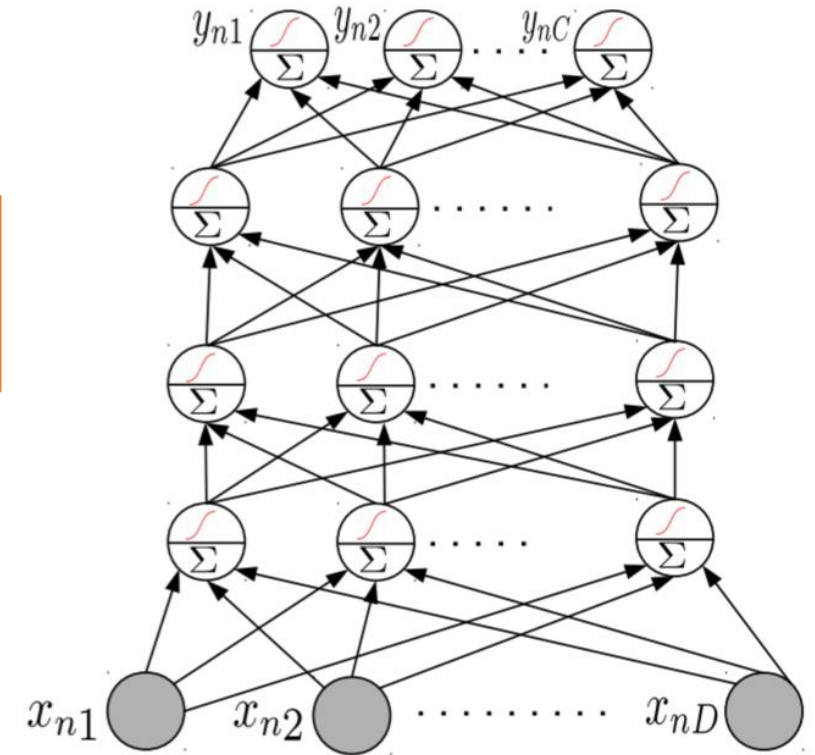


Neural Nets: Some Aspects



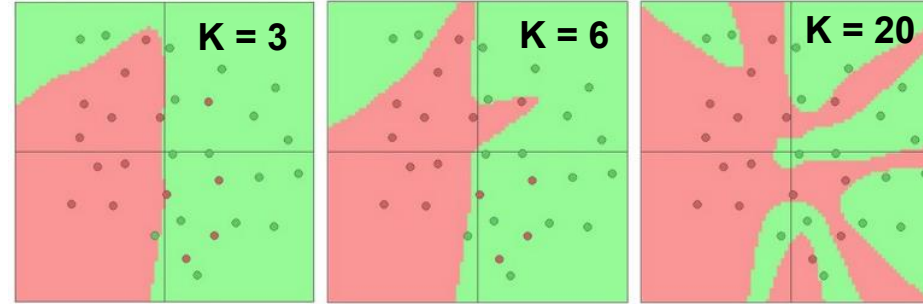
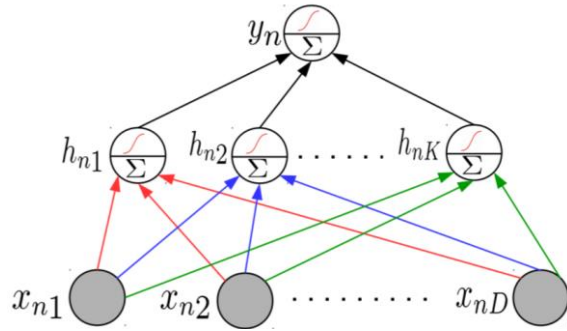
- Much of the magic lies in the hidden layers
- Hidden layers learn and detect good features
- Need to consider a few aspects
 - Number of hidden layers, number of units in each hidden layer
 - Why bother about many hidden layers and not use a single very wide hidden layer (recall [Hornik's universal function approximator theorem](#))?
 - Complex networks (several, very wide hidden layers) or simpler networks (few, moderately wide hidden layers)?
 - Aren't deep neural network prone to overfitting (since they contain a huge number of parameters)?

Choosing the right NN architecture is important and a research area in itself. [Neural Architecture Search \(NAS\)](#) is an automated technique to do this



Representational Power of Neural Nets

- Consider a single hidden layer neural net with K hidden nodes



- Recall that each hidden unit “adds” a simple function to the overall function
- Increasing K (number of hidden units) will result in a more complex function
- Very large K seems to overfit (see above fig). Should we instead prefer small K ?
- No! It is better to use large K and regularize well. Reason/justification:
 - Simple NN with small K will have a few local optima, some of which may be bad
 - Complex NN with large K will have **many local optimal, all equally good** (theoretical results on this)
- We can also use multiple hidden layers (each sufficiently large) and regularize well

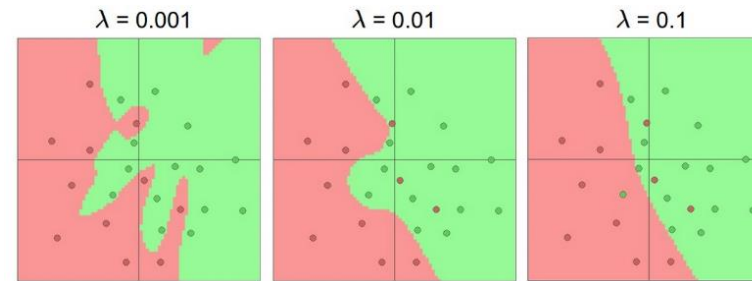
Preventing Overfitting in Neural Nets

Various other tricks, such as weight sharing across different hidden units of the same layer (used in [convolutional neural nets](#) or CNN)

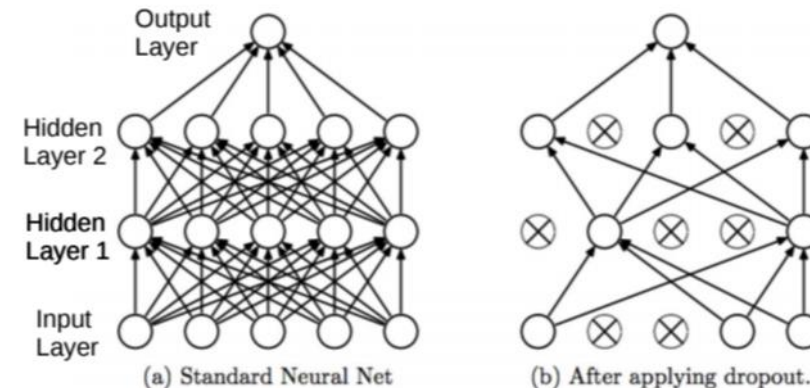
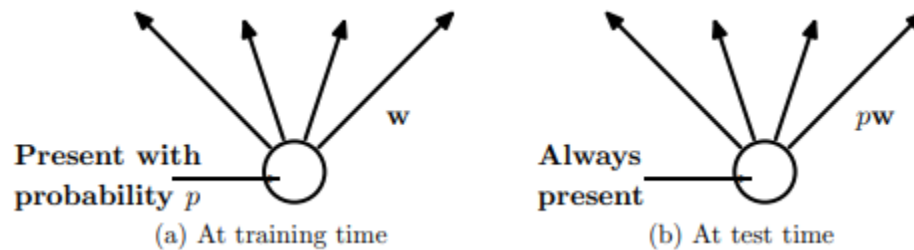


- Neural nets can overfit. Many ways to avoid overfitting, such as
 - [Standard regularization](#) on the weights, such as ℓ_2 , ℓ_1 , etc (ℓ_2 reg. is also called [weight decay](#))

Single Hidden Layer NN with K = 20 hidden units and L2 regularization



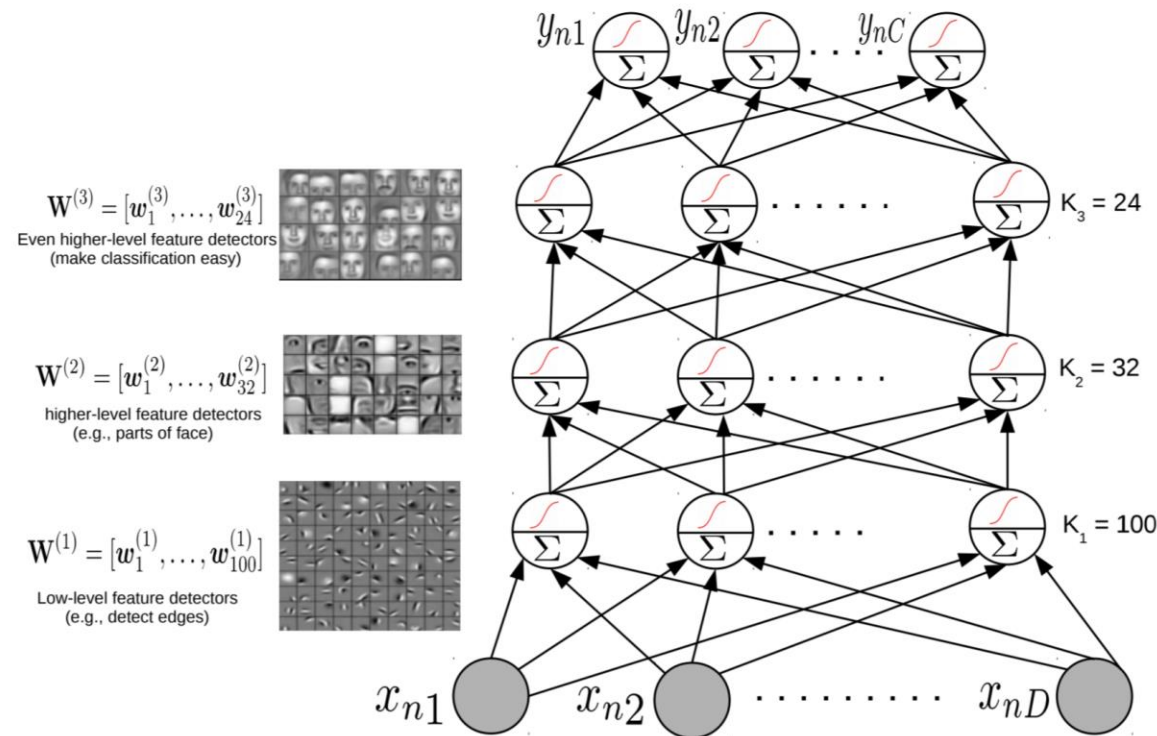
- [Early stopping](#) (traditionally used): Stop when validation error starts increasing
- [Dropout](#): Randomly remove units (with some probability $p \in (0,1)$) during training



Wide or Deep?



- While very wide single hidden layer can approx. any function, often we prefer many, less wide, hidden layers



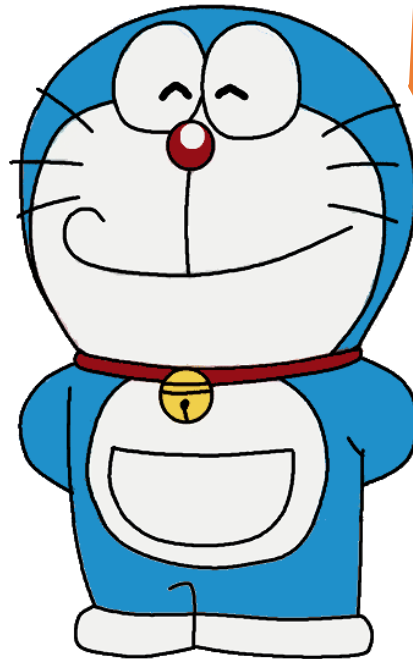
- Higher layers help learn more directly useful/interpretable features (also useful for compressing data using a small number of features)



Deep Neural Nets: Some Comments

- Highly effective in learning good feature rep. from data in an “end-to-end” manner
- The objective functions of these models are **highly non-convex**
 - But fast and robust non-convex opt algos exist for learning such deep networks
- Training these models is computationally very expensive
 - But GPUs can help to speed up many of the computations

Any question?



Readings for you:

- [Visual Introduction to Linear Regression](#)
- [MML Book](#)
- [Dive into Deep Learning](#)